



PDF Download
3694715.3695942.pdf
29 March 2026
Total Citations: 20
Total Downloads: 2573



Published: 15 November 2024

Citation in BibTeX format

SOSP '24: ACM SIGOPS 30th Symposium
on Operating Systems Principles
November 4 - 6, 2024
TX, Austin, USA

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3694715.3695942>

RESEARCH-ARTICLE

Autobahn: Seamless high speed BFT

NEIL GIRIDHARAN, University of California, Berkeley, Berkeley, CA, United States

FLORIAN SURI-PAYER, Cornell University, Ithaca, NY, United States

ITTAI ABRAHAM, Intel Corporation, Santa Clara, CA, United States

LORENZO ALVISI, Cornell University, Ithaca, NY, United States

NATACHA CROOKS, University of California, Berkeley, Berkeley, CA, United States

Open Access Support provided by:

Intel Corporation

Cornell University

University of California, Berkeley



Autobahn: Seamless high speed BFT

Neil Giridharan*
UC Berkeley

Florian Suri-Payer*
Cornell University

Ittai Abraham
Intel Labs

Lorenzo Alvisi
Cornell University

Natacha Crooks
UC Berkeley

Abstract

Today’s practical, high performance Byzantine Fault Tolerant (BFT) consensus protocols operate in the partial synchrony model. However, existing protocols are inefficient when deployments are indeed *partially* synchronous. They deliver either low latency during fault-free, synchronous periods (*good intervals*) or robust recovery from events that interrupt progress (*blips*). At one end, traditional, view-based BFT protocols optimize for latency during good intervals, but, when blips occur, can suffer from performance degradation (*hangovers*) that can last beyond the return of a good interval. At the other end, modern DAG-based BFT protocols recover more gracefully from blips, but exhibit lackluster latency during good intervals. To close the gap, this work presents Autobahn, a novel high-throughput BFT protocol that offers both low latency and *seamless* recovery from blips. By combining a highly parallel asynchronous data dissemination layer with a low-latency, partially synchronous consensus mechanism, Autobahn (i) avoids the hangovers incurred by traditional BFT protocols and (ii) matches the throughput of state of the art DAG-based BFT protocols while cutting their latency in half, matching the latency of traditional BFT protocols.

CCS Concepts: • Computer systems organization → Dependable and fault-tolerant systems and networks; • Security and privacy → Distributed systems security.

Keywords: consensus, Byzantine fault tolerance, blockchains, distributed systems

ACM Reference Format:

Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless high speed BFT. In *ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP ’24)*, November 4–6, 2024, Austin, TX, USA. ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/3694715.3695942>

1 Introduction

This work presents Autobahn, a Byzantine Fault Tolerant (BFT) state machine replication (SMR) protocol that sidesteps current tradeoffs between low latency, high throughput, and robustness to faults.

BFT SMR offers the appealing abstraction of a centralized, trusted, and always available server, even as some replicas

misbehave. Consensus protocols implementing this abstraction [23, 54, 60] are at the core of recent interest in decentralized systems such as blockchains, central-bank endorsed digital currencies [4, 5], and distributed trust systems. CCF, a Trusted Execution Environment (TEE) enabled BFT system [2], for instance, is used to power Azure SQL’s integrity functionality [1]. These protocols must offer (i) high throughput, (ii) low end-to-end latency, and (iii) robustness to both faults and changing network conditions; all whilst being simple enough to build and deploy.

Today’s popular (and actually deployed) BFT consensus protocols operate primarily in the *partial synchrony model* [31], introduced to get around the impossibility of a safe and live solution to consensus in an asynchronous system [34]. Partially synchronous protocols are always safe, but provide no liveness or performance guarantees before some “magic” *Global Stabilization Time* (GST), after which synchrony (all messages are received within a timebound Δ) is expected to hold forever on; they optimize for latency *after* GST holds. Deployed systems simulate this model using exponentially increasing timeouts – pragmatically, they declare GST attained and Δ found once timeouts are large enough that they are not being violated. In this regime, user requests (transactions) are guaranteed to commit.

In practice, however, this happy regime only holds intermittently. GST is an elegant fiction, as real deployments are subject to unpredictable replica and network failures, network adversaries (DDoS/route hijacking), latency fluctuations, link asymmetries, etc. that can cause periods in which progress stalls (*blips*). These begin when a timeout is violated and end once timeouts are again met (perhaps by adjusting the timeout length, or removing a presumed faulty leader). Unlike theoretical partial synchrony, which promises endless bliss after an initial (asynchronous) storm, *real* partial synchrony is instead piece-wise: periods where timeouts are met are separated by blips, during which progress is stalled.

Unfortunately, we find that existing partially synchronous protocols are not actually efficient under real partial synchrony: they have to choose between achieving low-latency when timeouts are met or remaining robust to blips.

Traditional BFT [23, 39, 45, 60] protocols and their multi-leader variants [40, 56, 57] optimize for performance after GST (*i.e.*, in the absence of timeout events). This design choice allows these protocols to minimize message exchanges in the common (synchronous) case. Unfortunately,

*Joint first authors, alphabetical.



we find that, after even a brief blip, these protocols cannot guarantee a graceful resumption of operations once synchrony returns; instead, they experience what we characterize as a *hangover*. Specifically, under sustained high load, the lack of progress caused by a blip (and the consequent loss of throughput) can generate large request backlogs that cause a degradation in end-to-end latency that persists well after the consensus blip has ended (Figs. 1,7,8). Thus, while on paper traditional BFT protocols enjoy low consensus latency, their poor resilience to blips can cause them to underperform under real-life partial synchrony.

Directed Acyclic Graph (DAG) based BFT, a newly popular class of consensus protocols [16, 26, 43, 48, 53, 54] takes an entirely different approach, steeped in an *asynchronous* system model. Asynchronous protocols optimize for worst-case message arrivals, and leverage randomness to guarantee progress without relying on timeouts. DAG systems employ as backbone a high throughput asynchronous data dissemination layer which, through a series of structured, tightly synchronized rounds, forms a DAG of temporally related data proposals. Replicas eventually converge on the same DAG and deterministically *interpret* their local view to establish a consistent total order. This approach yields excellent throughput and reduces the effects of blips on the system. Unfortunately, these benefits come at the cost of prohibitive latency; consequently, these protocols see little practical use. In pursuit of lower latency, recent (deployed) DAG protocols [53, 55] switch to partial synchrony and reintroduce timeouts, but still require several rounds of Reliable Broadcast [20] communication. For instance, Bullshark [54], the most popular partially synchronous DAG protocol, still requires up to 12 message delays (mds) to commit transactions.

This paper shows that, thankfully, these tradeoffs are not fundamental. We propose Autobahn, a new consensus protocol that minimizes hangovers without sacrificing low latency during synchronous periods. Autobahn’s architecture is inspired by recent DAG protocols: it constructs a highly parallel data dissemination layer that continues to make progress at the pace of the network (*i.e.*, at the lowest rate of transfer between correct nodes) even during blips. Atop, it carefully layers a traditional-style partially synchronous consensus mechanism that orders the stream of data proposals by committing concise state cuts. Like any partially synchronous protocol, consensus may fail to make progress during blips – however, upon return of progress, Autobahn can instantaneously commit the *entire* data backlog (with complexity independent from its size), minimizing the effect of hangovers.

Our results are encouraging (§6): Autobahn matches the throughput of Bullshark [54] while reducing its latency by more than half; and it matches the latency of HotStuff [60] without suffering the hangovers that HotStuff experiences.

This paper makes three core contributions:

- It formalizes two new notions – *hangovers* and *seamless* partial synchrony (§2) – to characterize the performance of partially synchronous protocols in practice.
- It revisits the set of system properties required to design a BFT consensus protocol that avoids protocol-induced hangovers (*i.e.* it is seamless).
- It presents the design of Autobahn, a novel *seamless* partially synchronous BFT consensus protocol that offers high throughput and low latency.

2 Partial Synchrony: Theory meets Practice

The partial synchrony model is an appealing theoretical framework for reasoning about liveness in spite of the seminal FLP impossibility result [34]. Real systems can, in principle, approximate this model using timeouts: they exponentially increase the value of the timeout until they enter a regime where timeout violations subside, and progress becomes possible, as if the network had indeed reached its Global Stabilization Time. Unfortunately, timeouts are notoriously hard to set [46, 53, 59]: too low of a timeout, and even common network fluctuations stop progress; too high, and the protocol is slow to respond to faults.

In practice, timeout violations are simply inevitable. Wide area network (WAN) latencies are unpredictable, can vary by order of magnitudes [41], and many RPCs have been shown to suffer from high tail latency [27, 51]. The operating context for today’s blockchain systems is even more challenging. Participants in these systems can have highly asymmetric network and hardware resources, and widely different deployment experiences, all factors that can greatly impact how well they can keep up with requests. These asymmetries complicate timeout configurations and result in frequent timeout events [7].

More fundamentally, accounting for the possibility of repeated timeout violations is simply part of the job description for any system that aims to be resilient to Byzantine faults. Malicious replicas may, for example, intentionally fail to send a required message, or a network adversary may target correct participants via DDoS/route hijacking attacks. Indeed, recent work, for instance, demonstrates that targeted network attacks on leaders are surprisingly easy to mount and drastically hurt performance [38].

Whatever the cause may be, the corrective actions that BFT SMR protocols take in response to a timeout violation (*e.g.*, electing a new consensus leader) can lead to *blips*, *i.e.*, periods during which progress stalls. Practical deployments are thus truly *partially* synchronous. Messages will not trigger timeouts for a while, but these periods of synchrony will be regularly interrupted by blips. To properly assess the performance of a partially synchronous system, we should consider how it behaves *at all times*, during periods of synchrony, but also after recovering from blips. To this effect, we introduce

the dual notions of *hangovers* and *seamlessness*. Informally, hangovers consist of performance degradations triggered by a blip that persist once the blip is over; we say that a partially synchronous protocol is seamless if it does not suffer from *protocol-induced* hangovers (defined in §2.1 below).

2.1 Hangovers & Seamlessness

Following Aardvark [24] and Abraham et al. [12] we say that an interval is *good* if the system is synchronous (w.r.t. some implementation-dependent timeout on message delay), and the consensus process is led by a correct replica. Intuitively, good intervals capture the periods during which progress is guaranteed; all non-good intervals are blips. Additionally, we consider an interval to be *gracious* [24] if it is good, and *all* replicas are correct. This distinction is helpful to reason about specific optimizations, such as so-called *fast paths* in the protocol. Given this, we define:

Definition 1 A *hangover* is any performance degradation caused by a blip that persists beyond the return of a good interval.

Hangovers impact both throughput and latency. We characterize a throughput hangover as the duration required to commit all outstanding transactions issued before the start of the next good interval. If the provided load is below the system’s steady-state throughput, the system will eventually recover all throughput “lost” during the blip.¹ Delayed throughput recovery in turn increases latency, even for transactions issued *during* a good interval that follows a blip. Before these transactions can commit, they must first wait for the backlog formed during the blip to commit. Hangovers can not only cause immediate latency spikes, but also make the system more susceptible to future blips. Good intervals do not last forever, and unresolved hangovers can add up—and continue indefinitely.

In contrast, in a hangover-free system, all submitted transactions whose progress is interrupted by a blip, commit instantaneously when progress resumes; further, these are the only transactions that experience a hike in latency, and for no more than the remaining duration of the blip.

Some hangovers are unavoidable, *e.g.*, those due to insufficient network bandwidth or to message delays. No consensus protocol, not even asynchronous ones, can provide progress beyond the pace of the network: without data, there isn’t anything to agree on! Consider, for instance, the case of a network partition that causes ten thousand 1MB transactions to accumulate; it will take at least 80 seconds, on a 1Gbit link, for them to be propagated once the partition resolves.

Other hangovers, however, are the result of suboptimal system design, where protocol logic (timeouts, commit rule, etc) introduces unnecessary delays. We refer to these hangovers as *protocol-induced*. As we describe further in §2.2,

¹If the load is equal or greater to the throughput, then the lost committed throughput (or *goodput*) can never be recovered.

the most common type of protocol-induced hangovers for consensus is the artificial coupling of data dissemination with the protocol’s ordering logic.

Thankfully, protocol-induced hangover are not inevitable and can be side-stepped through careful protocol design. Perhaps obviously, a protocol should accomplish this *without* making itself more susceptible to blips; a protocol that introduces *more blips* may implicitly introduce hangovers, or lose progress entirely.

We say that a consensus protocol is *seamless* if it behaves optimally in the face of blips. Formally:

Definition 2 A *partially synchronous* system is *seamless* if (i) it experiences no protocol-induced hangovers, and (ii) it does not introduce any mechanisms² that make the protocol newly susceptible to blips.

Seamlessness is a desirable property for any system, but especially relevant to BFT systems. A Byzantine leader can easily cause a blip: it can simply fail to propose. If this blip causes a hangover, it will continue to impact the performance of the system even after the leader has been replaced. This is clearly undesirable, especially because, after all, BFT protocols should be robust to Byzantine faults [24].

2.2 Existing Approaches

Traditional BFT protocols [23, 60] are not seamless. They tightly couple data dissemination and ordering: for each batch of transactions on which it seeks to achieve consensus, the protocol’s current leader jointly disseminates the data associated with the batch and proposes how this batch should be ordered relative to other batches. Consensus is then reached on each batch independently. If the consensus logic stalls, *e.g.*, because of a Byzantine leader triggering a timeout, so does data dissemination. When a good interval resumes, the time necessary to commit (*i.e.*, transmit, verify and order) the backlog of transactions accumulated during the blip will thus be on the order of the backlog’s size.

Take for instance, the behavior of HotStuff (HS) [60], a state of the art BFT protocol at the core of the former Diem blockchain [17]. We use a 1s (doubling) timeout (the default in many companies [7]), and trigger a three second blip by simulating a single leader failure; because HS pipelines rounds and eagerly rotates leaders, a single failure can cause two timeouts to trigger (§6). After the blip ends, HS, burdened with individually disseminating and processing the transaction batches accumulated in its backlog, suffers from a hangover that is 30% longer than the original blip itself! (Figure 1).

Simply proposing larger batches is not a viable option for reducing the number of batches in the backlog, as it comes at the cost of increased latency in the common case. A naive decoupling of data dissemination and consensus ordering logic (by proposing only a batch digest) [15, 23, 26] can help scale throughput, but is not robust: (i) malicious proposers

²Beyond the timeouts that are, of course, necessary for liveness.

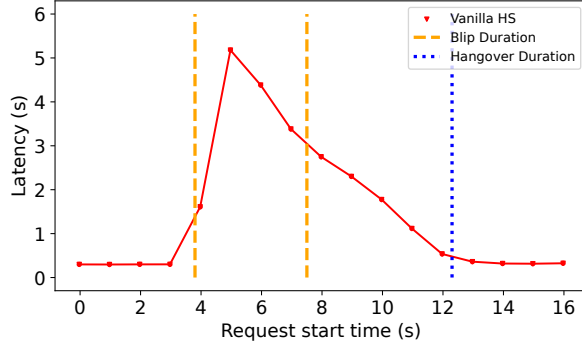


Figure 1. Latency Hangover in HotStuff [60].

may overwhelm correct replicas with data, and (ii) replicas that are out of sync must fetch missing data (data synchronization) before voting, which is on the timeout-critical path of consensus. We demonstrate empirically in §6 that such synchronization effects are common even in the absence of blips, and restrict throughput scalability.

Recent partially-synchronous DAG-based protocols [26, 54], in contrast, come *close* to seamlessness. These protocols decouple data dissemination from the consensus ordering logic, and construct an asynchronously growing DAG consisting of batches of transactions that are chained together. The construction leverages Reliable Broadcast (RB) to generate proofs that transitively vouch for the availability of those batches and of all topologically preceding batches in the DAG. The consensus ordering logic can then commit entire backlogs with constant cost by proposing only a single batch. Commit complexity is thus *independent* of blip duration.

Current DAG protocols allow data dissemination to proceed at the pace of the network, rather than that of consensus, but do so at the price of slowing down consensus, and thus fall short of seamlessness. In particular, they still place data synchronization on the timeout-critical path (before voting), which may introduce protocol-generated blips; further, they need to recursively traverse the DAG backwards to infer all topologically preceding data. This may require fetching data for multiple sequential rounds, delaying commit.³

Besides, existing DAG protocols exhibit during *good* intervals up to twice the latency of traditional BFT protocols. Consensus safety is hardwired into the structure of the DAG, and establishing it requires up to four rounds of RB (each comprising three message delays) – for a total of 12 message delays.

This paper asks: is it possible to develop a low-latency and high-throughput *seamless* protocol? We answer this question in the positive with Autobahn, a novel BFT protocol. Autobahn instantly commits all backlogged proposals upon returning from a blip, and without making consensus more susceptible to blips (thus achieving seamlessness).

³In practice, DAGs minimize worst-case synchronization by forcing (a supermajority of) replicas to advance through rounds in lock-step.

3 Autobahn: Overview

Autobahn comprises two logically distinct layers: a horizontally scalable data dissemination layer that always makes progress at the pace of the network; and a low-latency, partially synchronous consensus layer that tries to reach agreement on snapshots of the data layer. Decoupling consensus from data dissemination is not unique to Autobahn [15, 23, 26]; its core innovation lies in more cleanly separating the responsibility of each layer while carefully orchestrating their interaction. Seamlessness requires two key properties:

- *Responsive transaction dissemination.* The data dissemination layer should proceed at the pace of the network. We say that a transaction has been successfully disseminated once it is delivered by one correct replica.
- *Streamlined Commit.* During good intervals, all successfully disseminated proposals should be committed in bounded time and with cost and latency independent of the number of transactions to commit.

Implementing a responsive data dissemination layer may appear straightforward: simply broadcast all transactions to all replicas in parallel. This naive strategy unfortunately makes it extremely challenging to implement a consensus layer that streamlines commit. The consensus layer would have to individually propose (references to) all disseminated transactions, which would either require replicas to fetch and verify locally missing transactions (data synchronization) prior to voting in consensus, or risk breaking liveness in the presence of Byzantine proposers. Achieving seamlessness and low latency instead requires injecting the right amount of coordination in the data layer’s design: too little and the consensus layer cannot be designed to streamline commit; too much, and the good-case latency may spike. We find that the dissemination layer must specifically support *instant referencing*: it must allow the consensus layer to uniquely identify, and prove the availability of, the set of disseminated transactions. This must be done in constant time and space (w.r.t. to blip length), and without necessarily being in possession of all data. Note that the bounded time requirement implicitly requires that there be no data synchronization on the critical (voting) path of consensus, as this could lead to timeout violations, resulting in further blips (*non-blocking sync*). Moreover, any data syncing should finish by the time consensus commits to avoid delays that may cause hangovers (*timely sync*).

These principles shape Autobahn’s data dissemination and consensus layers.

Data dissemination. All replicas in Autobahn act as proposers, responsible for disseminating transactions received by clients. Each replica, in parallel, broadcasts batches of transactions (data proposals) and constructs a local chain (a *data lane*) that implicitly assigns an ordering to all of its data proposals. This structure allows us to transitively prove the availability of all data proposals in a chain in constant time,

thus guaranteeing *instant referencing*. Data lanes grow independently of one another, subject only to load and resource capacity, and are agnostic to blips (guaranteeing responsiveness). Finally, Autobahn guarantees that all successfully disseminated data proposals will commit during good intervals (*reliable inclusion*); this holds true even for Byzantine proposers, and ensures that replicas cannot "wastefully" disseminate transactions without intending to commit them.

Consensus. The consensus layer exploits the structure of data lanes to, in a single shot, commit an arbitrarily large data lane state. Autobahn efficiently summarizes data lane state as a cut containing only each replica's latest proposal (its *tip*), using instant referencing to uniquely identify a lane's history. Consensus itself follows a classical (latency optimal) PBFT-style coordination pattern [23, 36] consisting of two round-trips; during gracious intervals, Autobahn's *fast path* commits in only a single round-trip. After committing to a tip cut, a replica can infer the respective chain-histories of each tip and synchronize on any locally missing data (some transactions in the cut may not have been replicated to all nodes). To ensure that data synchronization doesn't stall consensus progress, Autobahn carefully constructs voting rules in the data dissemination layer to allow synchronization to complete in parallel with agreement, and in a single round-trip.

Once a replica is in possession of the data included in each tip's history data, it deterministically interleaves the n data lanes to construct a single total order.

This design allows a single consensus proposal to (i) reach throughput that scales in the number of replicas, (ii) reach agreement with cost independent of the size of the data lanes, and (iii) preserve the latency-optimality of traditional BFT protocols. We describe the full Autobahn protocol in §5.

4 Model

Autobahn adopts the standard assumptions of prior BFT work [23, 54, 60], positing $n = 3f + 1$ replicas, at most f of which are faulty. We consider a participant (client or replica) correct if it adheres to the protocol specification; a participant that deviates is considered faulty (or Byzantine). We make no assumptions about the number of faulty clients. We assume the existence of a strong, yet static adversary, that can corrupt and coordinate all faulty participants' actions; it cannot however, break standard cryptographic primitives. Participants communicate through reliable, authenticated, point-to-point channels. We use $\langle m \rangle_r$ to denote a message m signed by replica r . We expect that all signatures and quorums are validated, and that external data validity predicates are enforced; we omit explicit mention of this in protocol descriptions.

Autobahn operates under partial synchrony [31]: it makes no synchrony assumptions for safety, but guarantees liveness only during periods of synchrony [34].

5 Autobahn: The Protocol

To describe Autobahn more thoroughly, we first focus on the data dissemination layer (§5.1), which guarantees responsive data dissemination, even during blips. We then focus on how the consensus layer (§5.2.1) leverages the data layer's lane structure to efficiently commit arbitrarily long data lanes with cost and latency independent of their length, thus seamlessly recovering from blips.

5.1 Data dissemination

Autobahn's data dissemination layer offers the necessary structure for the consensus layer to ensure a streamlined commit. It precisely provides to consensus the three properties of instant referencing, non-blocking sync and timely-sync. Crucially, it provides no stronger guarantee, as additional properties may cause tail latency to increase.

Lanes and Cars. In Autobahn, every replica acts as a proposer, continuously batching and disseminating incoming transactions. Each replica proposes new batches of these transactions (data proposals) at its own rate, and fully independently of other replicas — we say that each replica operates in its own *lane*, and as fast as it can.

Within its lane, each replica leverages a simple Propose and Vote message pattern: the proposer broadcasts data proposals to all other nodes, and replicas vote to acknowledge delivery. $f + 1$ vote replies constitute a Proof of Availability (PoA) that guarantees at least one correct replica is in possession of the data proposal and can forward it to others if necessary. We call this simple protocol pattern a *car* (Certification of Available Request), illustrated in Figure 2. Note that Autobahn, unlike most DAG-BFT protocols, does not force all data proposals to go through a round of *reliable broadcast* [26, 54], which must process $n - f$ votes in order to achieve non-equivocation. Reliable, non-equivocating broadcast is not necessary in Autobahn to achieve the desired data layer properties (§5.2).

A lane is made up of a series of cars that are chained together (Fig. 3). A proposer, when proposing a new data proposal, must include a reference to its previous car's data proposal. Similarly, replicas will vote for a car at position i if and only if its proposal references a proposal for car $i - 1$, which the replica has already received and voted on. This construction ensures that a successful car for block i transitively proves the availability of a car for all blocks 0 to $i - 1$ in this proposer's lane. Validating the head of the lane is thus sufficient to reference arbitrary lane state (enabling instant referencing) and confirm that it has been disseminated (necessary for non-blocking sync). Requiring replicas to always vote *in-order* further ensures that at least one correct replica is in possession of all proposals in this lane, allowing the consensus layer to (§5.2.2) to experience timely-sync.

Protocol specification. Each replica maintains a local view of all lanes. We call the latest proposal of a lane a *tip*; a tip is certified if its car has completed (a matching PoA exists).

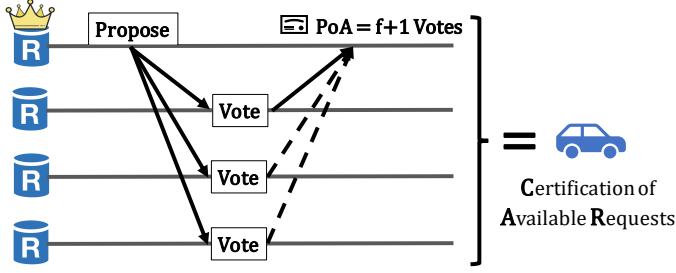


Figure 2. Car protocol pattern.

Algorithm 1 Data layer cheat sheet.

- 1: *car* $\triangleright$ *Propose* & *Vote* pattern
- 2: *PoA* $\triangleright$ $f+1$ matching VOTES for proposal
- 3: *pass* $\triangleright$ passenger: (proposal, Option(PoA))
- 4: *pos* $\triangleright$ car position in a lane (proposal seq no)
- 5: *lane* $\triangleright$ map: [*pos* $\rightarrow$ passenger]
- 6: *lanes* $\triangleright$ map: [replica $\rightarrow$ lane]
- 7: *tip* $\triangleright$ last passenger in a lane ($pos = lane.length$)

1: $P \rightarrow R$: Replica P broadcasts a new data proposal $PROP$.

P assembles a batch B of transactions, and creates a new data proposal for its lane l . It broadcasts a message $PROP := (\langle pos, B, parent \rangle_P, cert)$ where $pos = l.length + 1$ (the position of the proposal in the lane), $parent = h(l.tip.prop)$ (the hash of previous lane tip), and $cert = l.tip.PoA$ (the availability proof of the previous proposal).

2: $R \rightarrow P$: Replica R processes $PROP$ and votes.

R checks whether the received $PROP$ is valid: it checks that (i) it has already voted for the parent of this proposal ($h(lanes[P][pos-1].prop) == parent$) and (ii) it has not voted for this lane position before. R then stores the parent's PoA received as part of the message as the latest certified tip ($lanes[P][pos-1].PoA = cert$), and the current proposal as latest optimistic tip in $lanes[P][pos].prop = PROP$. Finally, it replies with a $VOTE := \langle dig = h(PROP), pos \rangle_R$. If R has not yet received the parent proposal, it buffers the request and waits.

3: $P \rightarrow R$: Replica P assembles VOTES and creates a PoA.

The proposer aggregates $f+1$ distinct matching $VOTE$ messages into a $PoA := (dig, pos, \{\sigma_r\})$, which it will include in the next car; if no new batch is ready then P can choose to broadcast the PoA immediately.

Cars enforce only light (but sufficient) structure on the data layer. The number and timing of cars in each lane may differ; they depend solely on the lane owner's incoming request load and connectivity (bandwidth, point-latencies). Cars also do not preclude equivocation. While lanes governed by correct replicas will only ever consist of a single chain, Byzantine lanes may fork arbitrarily. This is by design: non-equivocation is not a property that needs to be enforced at the data layer and causes unnecessary increase in tail latency.

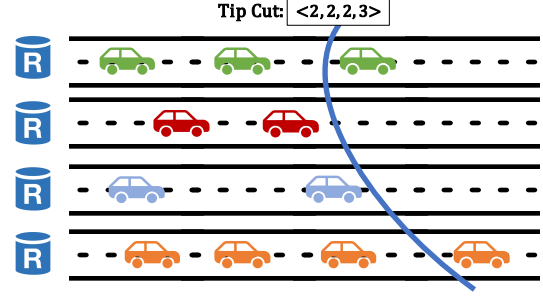


Figure 3. Autobahn parallel data lanes; example cut.

5.2 From Lanes to Consensus

As stated, Autobahn's data dissemination layer guarantees responsive data dissemination, instant referencing, and non-blocking, timely sync, but intentionally provides no consistency guarantee on the state observed by correct replicas. For instance, different replicas may observe inconsistent lane states (neither a prefix of another).

The role of the consensus layer in Autobahn is then to reconcile these views and to totally order all certified data proposals. To ensure seamless recovery from blips, the commit process should be *streamlined*: the consensus layer should be able to commit, in a single shot, an arbitrarily large number of certified data proposals. Unlike traditional BFT consensus protocols, that take as input a batch of transactions, a *consensus proposal* in Autobahn consists of a snapshot *cut* of all data lanes. Specifically, a consensus proposal contains a vector of n *certified* tip references, as shown in Figure 3.

This design achieves two goals: (i) committing a cut of all the lane states allows Autobahn to achieve consensus throughput that scales horizontally with the number of lanes (n total), while (ii) proposing certified lane tips allows Autobahn to commit an arbitrarily large backlog with complexity independent of its size. We discuss next how Autobahn reaches consensus on a cut (§5.2.1), and how it carefully exploits the lane structure to achieve seamlessness (§5.2.2).

5.2.1 Core consensus. Autobahn's consensus protocol follows a classic two-round (linear) PBFT-style [23] agreement pattern, augmented with a fast path that reduces latency to a single round (3mds) in gracious intervals. The protocol progresses as a series of slots: each slot s is assigned to a leader that can start proposing a new lane cut once slot $s-1$ commits. Within each slot, Autobahn follows a classical view based structure [23] – when a slot's consensus instance fails to make timely progress, replicas revolt, triggering a *View Change* and electing a new leader.

Each view consists of two phases: *Prepare* and *Confirm* (5mds total). The Prepare phase tries to achieve agreement within a view (non-equivocation) while the Confirm phase enforces durability across views (Fig. 4). During gracious intervals the *Confirm* phase can be omitted (Fast Path), reducing consensus latency down to a single phase (3mds).

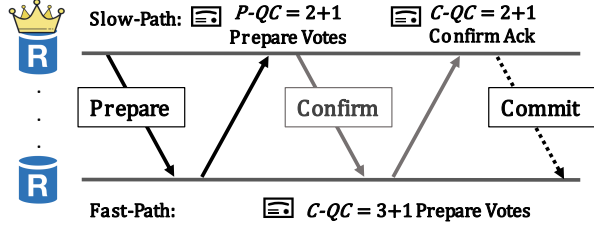


Figure 4. Core Consensus coordination pattern.

For simplicity, we first present our protocol for a single slot. We assume that all replicas start in view $v = 0$. We explain Autobahn’s view change logic in §5.3, and discuss in §5.4 how to orchestrate agreement for slots in parallel to reduce end-to-end latency. We defer formal proofs of correctness and seamlessness to the Appendix, A.

Algorithm 2 Consensus layer cheat sheet.

- 1: $\text{CommitQC}_s \triangleright$ Commit proof for slot s
- 2: $\text{TC}_{s,v} \triangleright$ View change proof for slot s , view v
- 3: Ticket $T_{s,v} \triangleright \text{COMMITQC}_{s-1}$ or $\text{TC}_{s,v-1}$
- 4: Proposal $P_{s,v} \triangleright$ Proposed lane cut. Requires $T_{s,v}$
- 5: $\text{prop} \triangleright$ map: slot $\rightarrow P_{s,v}$
- 6: $\text{conf} \triangleright$ map: slot $\rightarrow \text{PREPAREQC}_{s,v}$
- 7: $\text{last-commit} \triangleright$ map: lane $\rightarrow \text{pos}$
- 8: $\text{log} \triangleright$ total order of all user requests

Prepare Phase (P1).

- 1: **L $\rightarrow$ R:** Leader L broadcasts PREPARE.

A leader L for slot s begins processing (view $v=0$) once it has received a ticket $T \triangleq \text{COMMITQC}_{s-1}$ (Alg. 2), and “sufficiently” many new data proposals to propose a cut that advances the frontier committed in $s-1$; we coin this requirement *lane coverage*, and defer discussion of it to §5.2.3.

The leader broadcasts a message $\text{PREPARE} \triangleq (\langle P \rangle_L, T)$ that contains the *consensus proposal* P as well as the necessary ticket T . The proposal P itself consists of the slot number s , the view v , and a cut of the latest certified lane tips observed by L : $P \triangleq \langle s, v, [\text{lanes}[1].\text{tip.PoA}, \dots, \text{lanes}[n].\text{tip.PoA}] \rangle$.

- 2: **R $\rightarrow$ L:** Replica R processes PREPARE and votes.

R first confirms the PREPARE’s validity: it checks that (i) the ticket is for the right leader/slot, and (ii) R has not yet voted for this slot. R then stores a copy of the proposal ($\text{prop}[s] = P$), and responds with a $\text{PREP-VOTE} \triangleq \langle \text{dig} = h(P) \rangle_R$. Finally, if it has not locally received all tips (and their histories) included in P , R begins to asynchronously fetch all missing data (§5.2.2). This is possible thanks to the non-blocking sync guarantee of the data dissemination layer: R knows that the data will be available, and thus need not sync before voting.

- 3: **L:** Leader L assembles quorum of PREP-VOTE messages.

L waits for at least $n-f = 2f+1$ matching PREP-VOTE messages and aggregates them into a *Prepare Quorum Certificate*

(QC) $\text{PREPAREQC} \triangleq (s, v, \text{dig}, \{\text{PREP-VOTE}\})$. This ensures agreement *within* a view. No two PREPAREQCs with different *dig* can exist as all PREPAREQCs must intersect in at least one correct node, who will never vote more than once per view.

Fast Path. In the general case, the Prepare phase is insufficient to ensure that a particular proposal will be persisted across views. During a view change, a leader may not see sufficiently many PREP-VOTE messages to repropose this set of operations. In gracious intervals, however, the leader L can receive n such votes by waiting for a small timeout beyond the first $n-f$. A quorum of n votes directly guarantees durability across views (any subsequent leader will observe at least one PREP-VOTE for this proposal). No second phase is thus necessary. In this scenario, L upgrades the QC into a *fast commit quorum certificate* and proceeds directly to the Commit step, skipping the confirm phase: it broadcasts the COMMITQC , and commits locally.

- 4 (Fast): **L $\rightarrow$ R:** L upgrades to COMMITQC and commits.

Confirm Phase (P2). In the slow path, the leader L instead moves on to the Confirm phase.

- 1: **L $\rightarrow$ R:** Leader L broadcasts CONFIRM.

L forwards the $\text{PREPAREQC}_{s,v}$ by broadcasting a message $\text{CONFIRM} \triangleq \langle \text{PrepareQC}_{s,v} \rangle$ to all replicas.

- 2: **R $\rightarrow$ L:** Replica R receives and acknowledges CONFIRM.

R returns an acknowledgment $\text{CONFIRM-ACK} \triangleq \langle \text{dig} \rangle_R$, and buffers the PREPAREQC locally ($\text{conf}[s] = \text{PREPAREQC}$) as it may have to include this message in a later view change.

- 3: **L $\rightarrow$ R:** Leader L assembles COMMITQC and commits.

L aggregates $2f+1$ matching CONFIRM-ACK messages into a *Commit Quorum Certificate* $\text{COMMITQC} \triangleq (s, v, \text{dig}, \{\text{CONFIRM-ACK}\})$, and broadcasts it to all replicas.

5.2.2 Processing committed cuts. Recall that consensus proposals in Autobahn are cuts of data proposals, rather than simple batches of transactions. Upon committing, the replicas must then establish a (consistent) total order of proposals to correctly execute transactions. To help with this process, each replica maintains a log of all ordered transactions, and a map *last-commit* which tracks, for each lane, the position of the latest committed data proposal (Alg. 2).

Seamless data synchronization. To process a cut, a replica must first ensure that, for each lane, it is in possession of all data proposals that are transitively referenced by the respective tip (*tip history* for short). Otherwise, it must first acquire (*synchronize*) the missing data; in the worst-case, this may be the entire history. During a partial partition, for instance, lanes may grow without all correct replicas being aware, requiring them to later synchronize on missing data.

Autobahn exploits the lane structure to synchronize in parallel with consensus, and in just a single message exchange,

no matter the size of the history. First, certified tips allow replicas to transitively prove the availability of a lane history *without* directly observing the history. This allows replicas with locally inconsistent lane states to vote for consensus proposals without blocking (*non-blocking sync*), and moves synchronization off the timeout-critical path. If Autobahn employed only best-effort (non-certified) lanes, replicas would have to synchronize *before* voting on a consensus proposal to assert whether a tip (and its history) indeed exists. Such blocking increases the risk of timeouts undermining seamlessness. Second, in-order voting in the data layer enables *timely sync* by guaranteeing that there exists at least one correct replica that is in possession of *all* data proposals in a certified tip’s history.

To synchronize, a replica requests (from the replicas that voted to certify the tip) to SYNC all proposals in the tip’s history with positions greater than its locally last committed lane proposal (up to $\text{last-commit}[\text{tip.lane}].\text{pos} + 1$). By design, lane positions must be gap free, and proposals chained together; the requesting replica thus knows (i) whether a SYNC-REPLY contains the correct number of requested proposals, and (ii) whether the received proposals form a correct (suffix of the) history. By correctness of consensus, all correct replicas agree on the last committed lane proposal, and thus synchronize on the same suffix.

Creating a Total Order. Once a replica has fully synchronized the cut ($\langle \text{tips} \rangle$) committed in slot s , and all previous slots have been committed and processed, it tries to establish a total order across all “new” data proposals subsumed by the lane tips. A replica first identifies, for each lane l , the oldest ancestor start_l of $\text{tips}[l]$ with a position exceeding $\text{last}_l := \text{last-commit}[l]$ (i.e., $\text{start}_l.\text{pos} == \text{last}_l + 1$). Next, a replica updates the latest committed position for each lane ($\text{last-commit}[l] = \text{tips}[l].\text{pos}$), and appends to the log all new proposals (lane proposals from start_l to $\text{tips}[l]$) using some deterministic zipping function.

We note that the proposal committed at last_l may not (and need not) be the parent of start_l ; for instance, the replica governing l may have equivocated and sent multiple proposals for the same position. This does not affect safety as we simply ignore and garbage collect ancestors of start_l .

5.2.3 Governing Progress. Correct leaders in Autobahn only initiate a new consensus proposal once data lanes have made “ample” progress. We coin this requirement *lane coverage* – it is a tunable hyperparameter that governs the pace of consensus. Intuitively, there is no need for consensus if there is nothing new to agree on!

Choosing the threshold above which there are sufficiently many new data proposals to agree on requires balancing latency, efficiency, and fairness. Autobahn’s parameterizable lane coverage, unlike contemporary protocols that impose a fixed strategy [23, 26, 54, 60], offers deployments a *choice*: a system with heterogeneous lane capacities, for instance,

may opt to minimize latency by starting consensus for every new car; a resource conscious system, in contrast, may opt to wait for multiple new data proposals to be certified.

In Autobahn, we set coverage, as default, to require at least $n - f$ new tips. This ensures that at least half of the included tips belong to correct replicas. Tips, notably, may subsume a different number of proposals. Rather than enforce that replicas must make even progress (at times, elsewhere referred to as *chain quality* [26]), Autobahn opts to provide each replica the same *opportunity* for progress and thus can organically adapt to heterogeneous lane growth.

Lane coverage in Autobahn is best-effort and not enforced. From a performance perspective, a Byzantine leader that disregards coverage is equivalent to a crashed one; from a fairness perspective, it is equivalent to leaders in traditional BFT protocols that assemble batches at will. The next correct leader (in a good interval) will make up for any lost coverage.

Reliable Inclusion. Importantly, lane coverage does not cause Autobahn to discriminate against slow lanes. Unlike contemporary DAG-BFT protocols [26, 54] that are driven by the fastest $n - f$ replicas, and may ignore proposals of the f slowest, proposal cuts in Autobahn include *all* n lanes. Since correct replicas’ lanes never fork, all of their proposals are guaranteed to commit in a timely fashion (as soon as a correct leader receives a PoA that subsumes them).

Further, Autobahn’s consensus layer ensures that *all* certified proposals—including those of Byzantine replicas—are reliably committed. Since lane growth requires that at least one correct replica be in possession of a PoA for history of the latest car, Byzantine proposers cannot continue to disseminate transactions without intending to commit them; thus effectively bounding the maximum amount of “waste” that correct replicas may receive from Byzantine actors. We defer a formal analysis to the Appendix (A.4).

5.3 View change

In the presence of a faulty leader, progress may stall. As is standard, Autobahn relies on timeouts in these situations to elect a new leader: replicas that fail to see progress revolt, and create a special *Timeout Certificate* (TC) ticket to trigger a *View Change* [23, 45]. Each view v (for a slot s) is mapped to one designated leader (e.g. using round-robin). To prove its tenure a leader carries a ticket $T_{s,v}$, corresponding to either (in $v = 0$) a COMMITQC_{s-1} ,⁴ or (in $v > 0$) a quorum of mutineers $TC_{s,v-1}$ from view $v - 1$.

PREPARE and CONFIRM messages contain the view v associated with the sending leader’s tenure. Each replica maintains a *current-view_s*, and ignores all messages (besides COMMITQC ’s) with smaller views; if it receives a valid message in a higher view v' , it buffers it locally, and reprocesses it once it reaches view v' . Per view, a replica sends at most one PREP-VOTE and CONFIRM-ACK.

⁴The view associated with a COMMITQC ticket is immaterial; by safety of consensus, all COMMITQC ’s in a slot s must have the same value.

A replica starts a timer for view v upon first observing a ticket $T_{s,v}$. Seamlessness alleviates the need to set timeouts aggressively to respond quickly to failures. Autobahn favors conservative timers for smooth progress in good intervals as the data layer continues progressing even during blips. The replica cancels the timer for v upon locally committing slot s or observing a ticket $T_{s,v+1}$. Upon timing out, it broadcasts a complaint message **TIMEOUT** and includes any locally observed proposals that could have committed.

1: $R \rightarrow R$: Replica R broadcasts **TIMEOUT**.

A replica R , whose timer t_v expires, broadcasts a message **TIMEOUT** $\equiv (s, v, \text{highQC}, \text{highProp})_R$. highQC and highProp are, respectively, the $\text{PREPAREQC}_{s,v'}$ ($\text{conf}[s]$) and $\text{proposal}_{s,v''}$ ($\text{prop}[s]$) with the highest views locally observed by R . R will ignore future **PREPARE** and **CONFIRM** messages in view v .

2: R Replica R forms TC and advances view.

A replica R accepts a **TIMEOUT** message if it has not yet advanced to a higher view ($\text{current-view}_R \leq \text{TIMEOUT.v}$) or already received a **COMMITQC** for that slot. In the latter case, it simply forwards the **COMMITQC** to the sender of **TIMEOUT**.

Replica R joins the mutiny for view v once it has accepted at least $f+1$ **TIMEOUT** messages (indicating that at least one correct node has failed to see progress) and broadcasts its own **TIMEOUT** message for v . This ensures that if one correct replica locally forms a TC, all correct replicas eventually will. Upon accepting $2f+1$ distinct timeouts for v a replica assembles a *Timeout Certificate* $\text{TC} \equiv (s, v, \{\text{TIMEOUT}\})$, and advances its local view $\text{current-view} = v+1$ and starts a timer for $v+1$. If R is the leader for $v+1$, it additionally begins a new **Prepare** phase, using the TC as ticket.

A leader L that uses a TC as ticket must recover the latest proposal that *could have* been committed at some correct replica. To do so, the leader replica chooses, as *winning proposal*, the greater of (i) the highest highQC contained in TC, and (ii) the highest proposal present $f+1$ times in TC; in a tie, precedence is given to the highQC . This two-pronged structure is mandated by the existence of both a fast path and a slow path in Autobahn. If a proposal appears $f+1$ times in TC, then this proposal may have gone fast-path as going fast-path requires n votes. Any later $n-f$ quorums is guaranteed to see at least $f+1$ such votes. Similarly, if a **COMMITQC** formed on the slow path (from $2f+1$ **CONFIRM-ACKs** of a **PREPAREQC**), at least one such **PREPAREQC** will be included in the TC (by quorum intersection). Finally, the leader repropose the chosen consensus proposal.

Put together, this logic guarantees that if in some view v a **COMMITQC** was formed for a proposal P , all consecutive views will only re-propose P .

3: $L \rightarrow R$: Leader L sends PREPARE_{v+1} .

Finally, the leader L broadcasts a message $\text{PREPARE} \equiv (\langle v+1, \text{tips} \rangle_L, \text{TC})$, where tips is the winning proposal derived

from TC; or L 's local certified lane cut if no winning proposal exists. Replicas that receive such a **Prepare** message validate TC, and the correctness of *tips*, and otherwise proceed with **Prepare** as usual.

5.4 Parallel Multi-Slot Agreement

Until now, we have described Autobahn as proceeding through a series of sequential slot instances. Unfortunately, lane proposals that narrowly "miss the bus" and are not included in the current slot's proposed consensus cut may, in the worst case, experience the sequential latency of up to two consensus instances (the latency of the consensus instance they missed in addition to the one necessary to commit).

To circumvent sequential wait-times, Autobahn, inspired by PBFT [23], allows the next slot instance to begin in parallel with the current one, without waiting for it to commit. We modify a slot ticket for s to no longer require a COMMITQC_{s-1} : the leader of slot s can begin consensus for slot s as soon as it receives the first PREPARE_{s-1} message (the view is immaterial), and has observed sufficiently many new tips to satisfy lane coverage.

Parallel consensus can avoid sequential delays entirely when instantiated with a stable leader (the same leader proposes multiple slots). When instantiated with rotating leaders, there remains a single message delay: the time to receive PREPARE_{s-1} . We adjust Autobahn as follows.

Managing concurrent instances. Autobahn maintains independent state for all ongoing consensus slot instances. Each instance has its own consensus messages, and is agnostic to all other processing. Once a slot has committed, it waits until all preceding slot instances have already executed to execute itself. Similarly, when ordering a slot's proposal (a cut of tips), Autobahn replicas identify and order only proposals that are new. Old tip positions are simply ignored. This filtering step is necessary as consecutive slots may propose non-monotonic cuts.

Adjusting view synchronization. To account for parallel, overlapping instances, replicas no longer begin a timer for slot s upon reception of a COMMITQC_{s-1} , but upon reception of the first $\text{PREPARE}_{(s-1,v)}$ message. Moreover, we modify the leader election scheme to ensure that different slots follow a different leader election schedule. Specifically, we offset each election schedule by f . Without this modification, the worst case number of sequential view changes to commit k successive slots would be $\frac{k*(f+1)}{2}$, as each slot s would have to rotate through the same faulty leaders to generate a ticket for $s+1$.

Bounding parallel instances During gracious intervals, the number of concurrent consensus instances will be small. As each consensus instance incurs less than five message delays, there should, in practice, be no more than three to four concurrent instances at a time. During blips, however, this number can grow arbitrarily large: new instances may keep starting (as they only need to observe a relevant **PREPARE**

message). Continuously starting (but not finishing) parallel consensus instances is wasteful; it would be preferable to wait to enter a good interval, as a single new cut can subsume all past proposals. Autobahn thus bounds the maximum number of ongoing consensus instances to some k by (re-) introducing COMMITQC tickets. To begin slot instance s , one must include a ticket COMMITQC _{$s-k$} . When committing slot s we can garbage collect the COMMITQC for slot $s-k$ as CommitQC _{s} transitively certifies commitment for slot $s-k$.

Discussion. Autobahn’s parallel proposals departs from modern chained-BFT protocols [36, 37, 60] which pipeline consensus phases (e.g. piggyback PREPARE _{$s+1$} on CONFIRM _{s}). We decide against this design for three reasons: (i) pipelining phases is not latency optimal as it requires at least two message delays between proposals, (ii) it introduces liveness concerns that require additional logic to circumvent [37], and (iii) it artificially couples coverage between successive slots, *i.e.*, coverage for slot 2 affects progress of slot 1.

5.5 Optimizations

Finally, we discuss a number of optimizations, that, while not central to Autobahn’s ethos, can improve performance.

5.5.1 Pipelining cars. Replicas in Autobahn disseminate a lane proposal at position pos only upon completing certificate for $pos-1$. This is not strictly necessary for seamlessness, and Autobahn can, akin to consensus (§5.4), be configured to pipeline cars. Doing so, however, introduces a tradeoff: pipelining cars increases the number of un-certified proposals in the system that might never commit (A.4). We choose not to include this optimization in our prototype.

5.5.2 Exploring un-certified tips. By default, Autobahn consensus pessimistically only proposes *certified* tips. This allows voting in the consensus layer to proceed without blocking, which is crucial for seamlessness (§5.2). Assembling and sharing certificates, however, imposes a latency overhead of three message exchanges in addition to the unavoidable consensus latency. This results in a best-case end-to-end latency of six message delays on the fast path, and eight otherwise. Autobahn employs two optimizations to reduce the latency between the dissemination of a data proposal and it being proposed as part of consensus (inclusion latency).

Leader tips. First, we allow a leader to include a reference to the latest proposal (dig, pos) it has broadcast as its own lane’s tip, even as the tip has not yet been certified. If a Byzantine leader intentionally does not disseminate data, yet includes the proposal as a tip, it is primarily hurting itself. Correct nodes will, at most, make a single extra data sync request before requesting a view change. Moreover, if the Byzantine leader ever wants to propose additional data, it will have to backfill the data for this proposal (in-order voting).

Optimistic tips. Second, Autobahn may choose to relax seamlessness and allow leaders to optimistically propose non-certified tip references (dig, pos) for lanes that have, historically, remained reliable.⁵ In gracious intervals, this optimization lowers inclusion latency to a single message exchange, which is optimal. However, this comes at the risk of incurring blocking synchronization and triggering additional view changes. Replicas that have not received a tip locally hold off from voting, and request the tip from the leader directly.

We note, that critical-path synchronization is only necessary for the tip itself (and thus has constant cost) – by design, all ancestors must be certified, and thus can be synchronized asynchronously. Further, synchronization is only necessary in view $v = 0$, or in subsequent views that did not select a winning proposal from a prior view. In all other cases, the selected proposal P included in PREPARE _{$v' > 0$} is already implicitly certified (at least $f+1$ replicas voted for P).

In some cases, holding off from voting may be overly pessimistic and can unnecessarily impede agreement: while an individual replica might not (yet) be in possession of a tip’s data, there might be sufficiently many (at least $f+1$) other replicas that are, and can guarantee availability; yet insufficiently many (at least $2f+1$) to reach a PREPAREQC. To account for this, Autobahn can refine the voting procedure by allowing replicas to explicitly cast a “weak” VOTE for agreement only, or a “strong” VOTE that asserts both agreement and local availability. A replica may cast both a weak and strong VOTE for the same value within a view. A PREPAREQC is valid if it consists of $2f+1$ VOTE’s (analogously $3f+1$ for a fast COMMITQC), at least $f+1$ of which must be strong; thus ensuring availability.

5.5.3 Other optional modifications. Autobahn can, if desired, be further augmented with several standard techniques. We discuss them briefly for completeness.

Signature aggregation. Autobahn, like others [39, 54, 60] can be instantiated with threshold (TH) signatures [18, 52] that aggregate matching vote signature of any Quorum Certificate (PoA, PrepareQC, CommitQC) into a single signature. This reduces Autobahn’s complexity per car and consensus instance (during good intervals) to $O(n)$; view changes require $O(n^2)$ messages [60].

All-to-all communication. Autobahn follows a linear-PBFT communication pattern [39] that allows it to achieve linear complexity (during good intervals) when instantiated with aggregate signatures. Autobahn can, of course, also be instantiated with all-to-all communication for better latency (but no linearity). In this regime, consensus consists of only 3 message exchanges (2mds on FastPath): replicas broadcast PREPARE-VOTES and CONFIRM-ACKS, and respectively assemble *PrepareQCs* and *ConfirmQCs* locally.

⁵Akin to recent works [25, 53], Autobahn can be augmented with simple reputation mechanisms to determine reliable lanes (B.1).

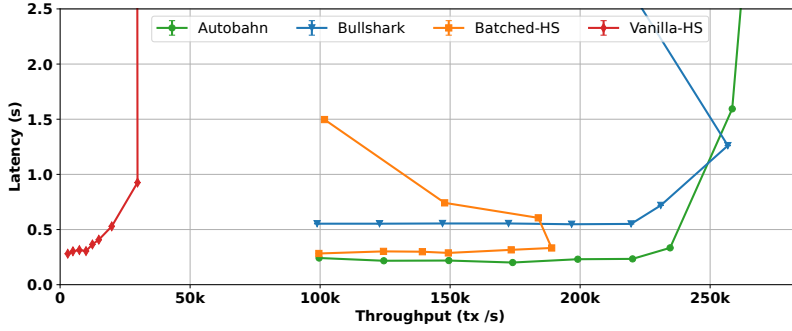


Figure 5. Throughput and Latency under increasing load.

6 Evaluation

Our evaluation seeks to answer three questions:

1. **Performance:** How well does Autobahn perform in gracious intervals? (§6.1)
2. **Scaling** How well does it scale as we increase n ? (§6.1)
3. **Blip Tolerance:** Does Autobahn make good on its promise of seamlessness? (§6.2)

We implement a prototype of Autobahn⁶ in Rust, starting from the open source implementation of Narwhal and Bullshark [33]. We use Tokio’s TCP [9] for networking, and RocksDB [8] for persistent storage. Finally, we use ed25519-dalek [3] signatures for authentication.

Baseline systems. We compare against open-source prototypes of HotStuff [60] and Bullshark [54, 55] which, respectively, represent today’s most popular traditional and DAG-based BFT protocol. We use the “batched” HotStuff (BatchedHS) prototype [32] that simulates an upper bound on achievable performance by naively separating dissemination and consensus. Replicas optimistically stream batches, and consensus leaders propose hashes of any received batches. This design is not robust in real settings: (i) Byzantine replicas and blips may exhaust memory. There is no guarantee that batches will be committed (no *reliable inclusion*) requiring the system to eventually drop transactions. (ii) Replicas must synchronize on missing batches before voting, which can cause blips (no *non-blocking, timely sync*). We also evaluate a standard version of HotStuff (VanillaHS) in which we modify BatchedHS to send batches only alongside consensus proposals of the issuing replica.

Both Bullshark [33] and Autobahn adopt the horizontally scalable worker layer proposed by Narwhal [26]. For a fair comparison with HotStuff we run only with a single, co-located worker; in this setup the Reliable Broadcast used by workers is unnecessary, so we remove it to save latency.

Experimental setup. We evaluate all systems on Google Cloud Platform (GCP), using an intra-US configuration with nodes evenly distributed in us-west1, us-west4, us-east1 and us-east5. We summarize RTTs between regions in Table 1. We use machine type t2d-standard-16[6] with 20GB

⁶<https://github.com/neilgiri/autobahn-artifact>

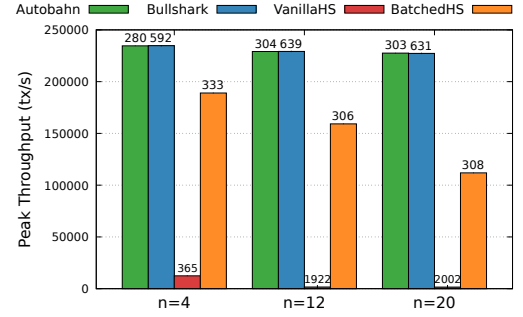


Figure 6. Peak throughput for varying n . Numbers atop bars show measured latency (ms).

RTT	us-east1	us-east5	us-west1	us-west4
us-east1	0.5	19	64	55
us-east5	19	0.5	50	57
us-west1	64	50	0.5	28
us-west4	55	57	28	0.5

Table 1. RTTs between regions (ms)

of SSD, and 10GB/s network bandwidth. A client machine (co-located in the same region as the replica) issues a constant stream of no-op transactions (tx), consisting of 512 random bytes [26, 54]; we denote as (input) load the total number of tx/s streamed by all clients.

We measure latency as the time between transaction arrival at a replica and the time it is ready to execute; throughput is measured as execution-ready transactions per second. Our measurements do not include client-to-replica (and replica-to-client) latency; this latency is constant, small (compared to the overall consensus latency), and independent of the protocol choice. Client request invocation requires only an in-data center ping latency to the local replica ($\approx 15\mu s$). Replies to the client incur the tail latency of the $f+1$ st closest replica (i.e., 19/28ms in our setup), or only the ping latency if the local replica is trusted (as is common in blockchains).

We set a batch size of 500KB (1000 transactions) across all experiments and systems, but allow consensus proposals to include/reference more than one batch if available; this *mini-batching* design [32, 33] allows replicas to organically reach larger effective batch sizes with reduced latency trade-off. All systems are run with rotating leaders; we set view timers to 1s. Each experiment runs for 60 seconds.

6.1 Performance in ideal conditions

Figure 5 shows the performance in a fault-free, synchronous setting, using $n = 4$ nodes. Autobahn’s throughput matches that of Bullshark (ca. 234K tx/s), but reduces latency by a factor of 2.1x (from 592ms to 280ms), edging out even BatchedHS (333ms) and VanillaHS (365ms). HotStuff and Bullshark require, respectively 7mids and up to 12 mids (10.5 mids average) to commit a transaction, while Autobahn reduces coordination to 4mids on the Fast Path, and 6mids otherwise (with optimistic tips). Both Autobahn and Bullshark significantly outperform both HotStuff variants for throughput by a factor of 1.23x (BatchedHS) and 15.6x (VanillaHS),

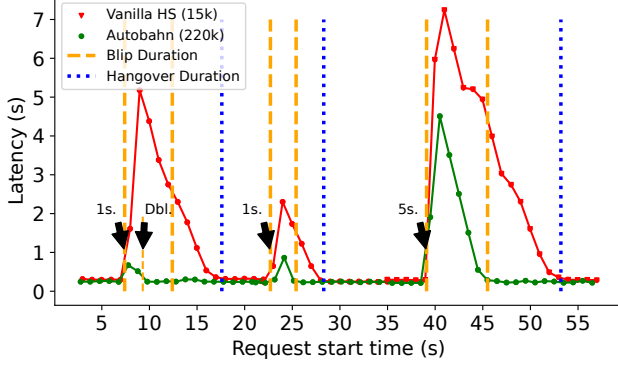


Figure 7. Leader failures.

and are bottlenecked on the cost of deserializing and storing data on disk. VanillaHS offers very low throughput (ca. 15K tx/s before latency noticeably rises) as it requires that replicas disseminate their own batches alongside consensus proposals. The system quickly bottlenecks on the (network and processing) bandwidth of one broadcast. BatchedHS scales significantly better (189K tx/s) than VanillaHS as it amortizes broadcasting cost. At high load, however, we find that it incurs a significant amount of data synchronization (resulting at times even in view changes) causing it to bottleneck; this corroborates findings in Narwhal [26] and demonstrates that this design is not robust in practice.

Fast Path/Optimistic Tips. To better understand the performance benefits of the fast path and optimistic tip optimizations, we evaluate Autobahn with the fast path deactivated and without the optimistic tip optimization. We omit the graph for space constraints. We observe a 40ms increase in latency when forced to always go “slow”. This increase is smaller than a cross-country RT: the fast path incurs higher tail latency due to the larger CommitQC ($|n|$) and because the leader ends up contacting non-local replicas, including those located across coasts. Similarly, we record an additional 33ms increase when proposing only certified tips. This follows directly from the need to contact $f+1$ replicas to form a PoA.

Scaling number of replicas. Figure 6 shows the peak throughput achievable as we increase the number of replicas. Both Autobahn and Bullshark scale gracefully as n increases; throughput remains bottlenecked on data processing, while latency is unaffected. In our setup, the total peak load P remains constant, but each replica proposes a smaller fraction as n grows since input load is balanced across replicas (P/n per replica).⁷ BatchedHS, in contrast, experiences a sizeable throughput degradation with growing n (−16% at $n = 12$, and −41% at $n = 20$) as replicas are increasingly likely to be out of sync and thus are forced to perform additional data synchronization. Finally, we observe that VanillaHS suffers a significant throughput-latency tradeoff: because each replica proposes unique batches, and batches are only

⁷If replicas were to instead propose at constant rates $r < P/n$, throughput would increase with n (up to the data processing bottleneck P).

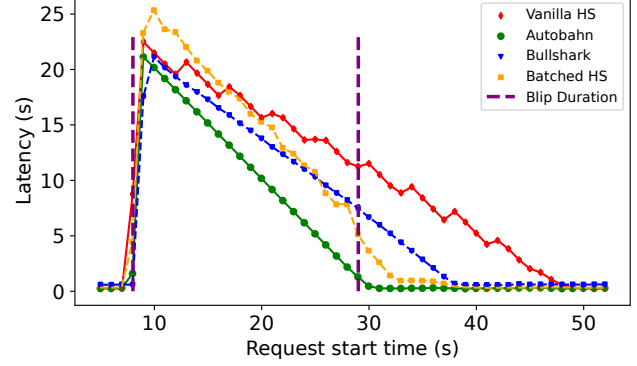


Figure 8. Partial partition.

proposed when a replica becomes the leader, latency grows proportionally to n . We highlight this tension by bounding latency to be at most 2s. In this setup, VanillaHS throughput diminishes sharply as n rises; it drops from 15K to only 1.5K tx/s as latency surges rapidly. VanillaHS is 1.3 times slower than Autobahn at $n=4$, but over 6.6 times slower at $n=20$.

6.2 Operating in *partial synchrony*

The previous section highlighted Autobahn’s good performance in the failure-free scenario: Autobahn achieves the best of both worlds. It matches Bullshark’s throughput while preserving HotStuff’s low latency. We now investigate whether Autobahn remains robust to blips. We use $n=4$.

Faulty Leader Blips. We first consider blips that result from a single leader failure. Bullshark and BatchedHS exhibit the same blip behavior as Autobahn, and we thus omit them for clarity. Figure 7 plots latency over time. Data points are averages over second-long windows. For the first two blips, we use a standard timeout of 1s [7]. HotStuff’s pipelined, rotating leader design, can, in the presence of a faulty leader, trigger not one but two timeouts. Because the votes for a proposal issued by a leader $L1$ are eagerly forwarded only to $L2$, a single failure of $L2$ can cause two timeouts to trigger: one for its preceding proposal, and one for its own. When instantiated with stable leaders, it can only generate a single blip. We thus consider both scenarios (respectively called Dbl, and 1s). VanillaHS (under 15K tx/s load) suffers from hangovers that persist for respectively 1.6x, 1.3x, and 1.2x of the blip duration beyond the good interval resuming. The system remains bottlenecked by data dissemination and delivery when trying to work off the transaction backlog. For the Dbl (3s) blip, for instance, the latency penalty not only exceeds the blip duration, for a maximum transaction latency of 6.2s, but remains as high as 1.3s after three seconds post-blip, and 700ms at four seconds post-blip (respectively 3.5x and 1.9x increase over the steady state latency of 365 ms). Autobahn (under 220K tx/s load), in contrast, immediately recovers from the hangover. This behavior continues to hold true for larger timeout values. For a timeout as high 5s (still realistic – Diem, Facebook’s former blockchain used

a timeout of 30s [7, 17]), VanillaHS’s hangover persisted for almost 7 seconds after the blip ended.

Partition Blips. Next, we consider a blip caused by a temporary network partition (20s) that isolates replicas into two halves (Fig. 8). The load is 15K tx/s (the max for VanillaHS). Autobahn experiences only a small hangover because it continues to disseminate data (replicas can reach $f + 1$ other replicas, enough to grow their lanes). Once the partition resolves, a single slot instantly commits the entire lane backlog, and replicas identify and request (in a single step) all missing data. It takes, respectively, about 1s to transfer and process all missing data (we are bottlenecked by bandwidth and delivery); this illustrates an unavoidable, non protocol-induced hangover. Transactions submitted during the partition period experience a latency penalty close to the remaining blip duration. Bullshark and BatchedHS require slightly longer to recover (both ca. 9s). BatchedHS optimistically continues to disseminate during the partition, but upon resolution must enforce a cap on mini-batch references per proposal to avoid excessive synchronization on the timeout-critical path. Bullshark cannot continue to disseminate during the partition because it must reach $2f + 1$ nodes to advance the DAG; after it resolves, however, its efficient dissemination can quickly recover the backlog. VanillaHS, in contrast, exhibits a large hangover, directly proportional to the blip duration.

7 Related Work

Autobahn draws inspiration from several existing works. Scalog [28] demonstrates how to separate data dissemination from ordering to scale throughput at low latencies (for crash failures). Narwhal [26] discusses how to implement a scalable and BFT robust asynchronous broadcast layer, a core tenet for achieving seamlessness. Autobahn’s consensus itself is closely rooted in traditional, latency-optimal BFT-protocols, most notably PBFT’s core consensus logic and parallelism [23], Zyzzyva’s fast path [39, 45], and Aardvark’s doctrine of robustness [24].

Motorizing Consensus Protocols. Autobahn’s architecture, by design, isn’t tied to a specific consensus mechanism, and thus can easily accommodate future algorithmic improvements (including advances in asynchronous protocols). Furthermore, existing consensus deployments may easily be *motorized* to enjoy both improved throughput and seamlessness without adopting the custom consensus protocol described in Autobahn: a blackbox consensus mechanism of choice may be augmented by adding lanes, replacing consensus proposals with cuts, and adopting the synchronization and ordering logic presented.

Traditional BFT protocols such as PBFT [23], HotStuff [60], and their respective variants [39, 45][36, 42] optimize for good intervals: they minimize latency in the common case, but guarantee neither progress during periods of asynchrony, nor resilience to hangovers. Multi-log frameworks [14, 40, 56, 57] partition the request space across multiple proposers

to sidestep a perceived leader bottleneck, and intertwine the sharded logs to arrive at a single, total order. They inherit both the good and the bad of their BFT building blocks: consensus has low latency, but remains prone to hangovers.

Directed Acyclic Graph (DAG) BFT protocols, as previously discussed [16, 26, 43, 48, 54], originate in the asynchronous model and enjoy both excellent throughput and network resilience; recent designs [44, 53, 55] make use of partial synchrony to improve consensus latency. *Certified* DAGs [26, 54] are close to achieving seamlessness (data synchronization on the timeout-critical path aside), but use Reliable Broadcast [19, 20] (RB) to implement the voting steps of traditional BFT consensus, resulting in high latencies. Bullshark [55], for instance, requires up to 4 rounds of RB (each comprised of 3 message exchanges) to commit, for a worst case total latency of 12mds (9mds when optimized by Shoal [53]).

Recent works propose (but do not empirically evaluate) uncertified DAGs [44, 47]. These designs aim to improve latency by replacing the RB in each round with best-effort-broadcast (BEB), but forgo seamlessness because they must synchronize (possibly recursively) on missing data before voting.

Star [30] adopts a construction similar to Autobahn. It uses n parallel proposers that create “weak-certificates” that only prove availability (equivalent to PoA), but retains a rigid round-based DAG-like construction. For each round it requires an external single-shot PBFT instance that commits $n - f$ weak-certificates; this design is prone to hangovers as the number of required consensus instances is directly proportional to blip duration.

DAG vs Lanes. Lanes in Autobahn loosely resemble the DAG structure, but minimize quorum sizes and eschew rigid dependencies across proposers. Notably, Autobahn proposes a cut of all n replica lanes; in contrast, DAGs can reliably advance proposals simultaneously from only $n - f$ replicas, and are thus subject to ignoring some proposals [54].

Asynchronous BFT consensus protocols [11, 26, 36, 46, 49] sidestep the FLP impossibility result [34] and guarantee liveness even during asynchrony by introducing randomization. They typically use a combination of reliable broadcast (RB) [19, 20] and asynchronous Byzantine Agreement (BA) to implement asynchronous Byzantine atomic broadcast [29, 35, 49]. Unfortunately, these protocols require expensive cryptography and several rounds of message exchanges, resulting in impractical latency. The asynchronous systems closest related to Autobahn are Honeybadger [49] and Dumbo-NG [35]. Honeybadger [49] achieves high throughput by operating n instances of RB in parallel and trading off latency for large batch sizes; it uses n binary BA instances [10, 19, 22, 50] to select $n - f$ proposals that may commit. Dumbo-NG [35] separates data dissemination from consensus to ameliorate batching/latency trade-offs. It constructs parallel chains of RB and layers a multi-valued BA protocol [11, 21] atop; this architecture closely resembles the spirit of Autobahn, but incurs significantly higher latency.

8 Conclusion

This work presents Autobahn, a novel partially synchronous BFT consensus protocol that is actually robust to *partial synchrony*. Autobahn achieves seamlessness, while simultaneously matching the throughput of DAG-based BFT protocols and the latency of traditional BFT protocols. Drive safe!

Acknowledgments

We are grateful to Brad Karp (our shepherd), and the anonymous reviewers for their thorough and insightful comments. This work was supported in part by the NSF grants CSR-17620155, CNS-CORE 2008667, CNS-CORE 2106954, a Sui Research Award, support from the Initiative for Cryptocurrencies and Contracts (IC3) as well as gifts from Accenture, AMD, Anyscale, Google, IBM, Intel, Mohamed Bin Zayed University of Artificial Intelligence, Samsung SDS, SAP, and VMware.

References

- [1] [n. d.]. *Azure Confidential Ledger*. <https://azure.microsoft.com/en-gb/products/azure-confidential-ledger> (last accessed on 09/23/24).
- [2] [n. d.]. *Confidential Consortium Framework*, Microsoft. <https://ccf.microsoft.com/> (last accessed on 09/23/24).
- [3] [n. d.]. Dalek elliptic curve cryptography. <https://github.com/dalek-cryptography/ed25519-dalek> (last accessed on 09/23/24).
- [4] [n. d.]. Digital Euro. https://www.ecb.europa.eu/euro/digital_euro/html/index.en.html (last accessed on 09/23/24).
- [5] [n. d.]. Digital euro explained: Is the digital euro based on blockchain? <https://cointelegraph.com/learn/the-digital-euro-project-how-will-it-impact-the-current-financial-system> (last accessed on 09/23/24).
- [6] [n. d.]. Google Cloud Platform (GCP) - general purpose machines. https://cloud.google.com/compute/docs/general-purpose-machines#t2d_machines (last accessed on 09/23/24).
- [7] [n. d.]. Private communications with researchers at two of the leading DAG-based blockchain companies (Aptos, Mysten), and formerly of Facebook Novi. March 2024.
- [8] [n. d.]. RocksDB, version 0.16.0. <https://rocksdb.org/> (last accessed on 09/23/24).
- [9] [n. d.]. Tokio, version 1.5.0. <https://tokio.rs/> (last accessed on 09/23/24).
- [10] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. 2022. Efficient and Adaptively Secure Asynchronous Binary Agreement via Binding Crusader Agreement. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing*. 381–391.
- [11] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. 2019. Asymptotically Optimal Validated Asynchronous Byzantine Agreement. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing* (Toronto ON, Canada) (PODC '19). Association for Computing Machinery, New York, NY, USA, 337–346. <https://doi.org/10.1145/3293611.3331612>
- [12] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2021. Good-Case Latency of Byzantine Broadcast: A Complete Categorization. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing* (Virtual Event, Italy) (PODC'21). Association for Computing Machinery, New York, NY, USA, 331–341. <https://doi.org/10.1145/3465084.3467899>
- [13] Ittai Abraham and Alexander Spiegelman. 2022. Provable Broadcast. <https://decentralizedthoughts.github.io/2022-09-10-provable-broadcast/> (last accessed on 09/23/24).
- [14] Balaji Arun and Binoy Ravindran. 2022. Scalable byzantine fault tolerance via partial decentralization. 15, 9 (2022).
- [15] Vivek Bagaria, Sreeram Kannan, David Tse, Giulia Fanti, and Pramod Viswanath. 2019. Prism: Deconstructing the blockchain to approach physical limits. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 585–602.
- [16] Leemon Baird. 2016. The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. *Swirlds Tech Reports SWIRLDS-TR-2016-01, Tech. Rep* 34 (2016), 9–11.
- [17] Mathieu Baudet, Avery Ching, Andrey Chursin, George Danezis, François Garillot, Zekun Li, Dahlia Malkhi, Oded Naor, Dmitri Perelman, and Alberto Sonnino. 2019. State machine replication in the Libra Blockchain. *The Libra Association Technical Report* (2019).
- [18] Alexandra Boldyreva. 2002. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In *International Workshop on Public Key Cryptography*. Springer, 31–46.
- [19] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and Computation* 75, 2 (1987), 130–143.
- [20] Gabriel Bracha and Sam Toueg. 1985. Asynchronous consensus and broadcast protocols. *Journal of the ACM (JACM)* 32, 4 (1985), 824–840.
- [21] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*. Springer, 524–541.
- [22] Christian Cachin, Klaus Kursawe, and Victor Shoup. 2000. Random oracles in constant-time: practical asynchronous byzantine agreement using cryptography. In *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*. 123–132.
- [23] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation* (New Orleans, Louisiana, USA) (OSDI '99). USENIX Association, USA, 173–186.
- [24] Allen Clement, Edmund Wong, Lorenzo Alvisi, Mike Dahlin, and Mirco Marchetti. 2009. Making Byzantine Fault Tolerant Systems Tolerate Byzantine Faults. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation* (Boston, Massachusetts) (NSDI'09). USENIX Association, USA, 153–168.
- [25] Shir Cohen, Rati Gelashvili, Lefteris Kokoris Kogias, Zekun Li, Dahlia Malkhi, Alberto Sonnino, and Alexander Spiegelman. 2022. Be Aware of Your Leaders. In *Financial Cryptography and Data Security: 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers* (Grenada, Grenada). Springer-Verlag, Berlin, Heidelberg, 279–295. https://doi.org/10.1007/978-3-031-18283-9_13
- [26] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 34–50.
- [27] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80.
- [28] Cong Ding, David Chu, Evan Zhao, Xiang Li, Lorenzo Alvisi, and Robbert Van Renesse. 2020. Scalog: Seamless reconfiguration and total order in a scalable shared log. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 325–338.
- [29] Sisi Duan, Michael K. Reiter, and Haibin Zhang. 2018. BEAT: Asynchronous BFT Made Practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). Association for Computing Machinery, New York, NY, USA, 2028–2041. <https://doi.org/10.1145/3243734.3243812>
- [30] Sisi Duan, Haibin Zhang, Xiao Sui, Baohan Huang, Changchun Mu, Gang Di, and Xiaoyun Wang. 2024. Dashing and Star: Byzantine fault tolerance with weak certificates. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 250–264.
- [31] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)* 35, 2 (1988), 288–323.

- [32] Novi Facebook Research. [n.d.]. Hotstuff Implementation. <https://github.com/asonnino/hotstuff/commit/d771d4868db301bc5e3deaa915b5017220463f6> (last accessed on 09/10/24).
- [33] Novi Facebook Research. [n.d.]. Narwahl and Bullshark implementation. <https://github.com/asonnino/narwahl> (last accessed on 09/23/24).
- [34] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. 1985. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)* 32, 2 (1985), 374–382.
- [35] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2022. Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1187–1201.
- [36] Rati Gelashvili, Lefteris Kokoris-Kogias, Alberto Sonnino, Alexander Spiegelman, and Zhuolun Xiang. 2022. Jolteon and Ditto: Network-Adaptive Efficient Consensus with Asynchronous Fallback. In *Financial Cryptography and Data Security: 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers* (Grenada, Grenada). Springer-Verlag, Berlin, Heidelberg, 296–315. https://doi.org/10.1007/978-3-031-18283-9_14
- [37] Neil Giridharan, Florian Suri-Payer, Matthew Ding, Heidi Howard, Ittai Abraham, and Natacha Crooks. 2023. BeeGees: stayin’ alive in chained BFT. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing*. 233–243.
- [38] Giacomo Giuliani, Alberto Sonnino, Marc Frei, Fabio Streun, Lefteris Kokoris-Kogias, and Adrian Perrig. 2024. An Empirical Study of Consensus Protocols’ DoS Resilience. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1345–1360.
- [39] Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. 2019. SBFT: A Scalable and Decentralized Trust Infrastructure. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, USA, 568–580. <https://doi.org/10.1109/DSN.2019.00063>
- [40] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: resilient concurrent consensus for high-throughput secure transaction processing. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1392–1403.
- [41] Toke Høiland-Jørgensen, Bengt Ahlgren, Per Hurtig, and Anna Brunstrom. 2016. Measuring latency variation in the internet. In *Proceedings of the 12th International Conference on emerging Networking EXperiments and Technologies*. 473–480.
- [42] Mohammad M. Jalalzai, Jianyu Niu, and Chen Feng. 2020. Fast-HotStuff: A Fast and Resilient HotStuff Protocol. *CoRR* abs/2010.11454 (2020). [arXiv:2010.11454](https://arxiv.org/abs/2010.11454) <https://arxiv.org/abs/2010.11454>
- [43] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All you need is dog. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. 165–175.
- [44] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *37th International Symposium on Distributed Computing (DISC 2023)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [45] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2010. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Transactions on Computer Systems (TOCS)* 27, 4, Article 7 (Jan. 2010), 39 pages. <https://doi.org/10.1145/1658357.1658358>
- [46] Shengyun Liu, Wenbo Xu, Chen Shan, Xiaofeng Yan, Tianjing Xu, Bo Wang, Lei Fan, Fuxi Deng, Ying Yan, and Hui Zhang. 2023. Flexible Advancement in Asynchronous BFT Consensus. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 264–280.
- [47] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. 2024. BBKA-CHAIN: Low Latency, High Throughput BFT Consensus on a DAG. *Financial Cryptography and Data Security (FC’24)* (2024).
- [48] Dahlia Malkhi and Pawel Szalachowski. 2022. Maximal Extractable Value (MEV) Protection on a DAG. *arXiv preprint arXiv:2208.00940* (2022).
- [49] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The Honey Badger of BFT Protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (Vienna, Austria) (CCS ’16)*. Association for Computing Machinery, New York, NY, USA, 31–42. <https://doi.org/10.1145/2976749.2978399>
- [50] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. 2014. Signature-free asynchronous Byzantine consensus with $t < n/3$ and $O(n^2)$ messages. In *Proceedings of the 2014 ACM symposium on Principles of distributed computing*. 2–9.
- [51] Korakit Seemakhupt, Brent E Stephens, Samira Khan, Sihang Liu, Hassan Wassel, Soheil Hassas Yeganeh, Alex C Snoeren, Arvind Krishnamurthy, David E Culler, and Henry M Levy. 2023. A Cloud-Scale Characterization of Remote Procedure Calls. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 498–514.
- [52] Victor Shoup. 2000. Practical threshold signatures. In *Advances in Cryptology—EUROCRYPT 2000: International Conference on the Theory and Application of Cryptographic Techniques Bruges, Belgium, May 14–18, 2000 Proceedings 19*. Springer, 207–220.
- [53] Alexander Spiegelman, Balaji Aurn, Rati Gelashvili, and Zekun Li. 2024. Shoal: Improving DAG-BFT Latency And Robustness. *Financial Cryptography and Data Security (FC’24)* (2024).
- [54] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (Los Angeles, CA, USA) (CCS ’22)*. Association for Computing Machinery, New York, NY, USA, 2705–2718. <https://doi.org/10.1145/3548606.3559361>
- [55] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: The Partially Synchronous Version. *arXiv preprint arXiv:2209.05633* (2022).
- [56] Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolić. 2019. Mir-bft: High-throughput robust bft for decentralized networks. *arXiv preprint arXiv:1906.05552* (2019).
- [57] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. 2022. State machine replication scalability made simple. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 17–33.
- [58] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. 2021. Basil: Breaking up BFT with ACID (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 1–17.
- [59] Pasindu Tennage, Cristina Basescu, Lefteris Kokoris-Kogias, Ewa Syta, Philipp Jovanovic, Vero Estrada-Galananes, and Bryan Ford. 2023. QuePaxa: Escaping the tyranny of timeouts in consensus. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 281–297.
- [60] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (Toronto ON, Canada) (PODC ’19)*. Association for Computing Machinery, New York, NY, USA, 347–356. <https://doi.org/10.1145/3293611.3331591>

A Proofs

A.1 Safety

We show that all correct replicas commit, for every slot, the same proposal. It necessarily follows, that all correct replicas compute a consistent log. In the following, we prove agreement for a given slot s , and drop the slot number for convenience.

We first prove that, within a view v , no two conflicting proposals can be committed.

Lemma 1. *If there exists a PREPAREQC or COMMITQC for consensus proposal p in view v , then there cannot exist a PREPAREQC or a COMMITQC for consensus proposal $p' \neq p$ in view v .*

Proof. Suppose for the sake of contradiction there exists a PREPAREQC or COMMITQC for consensus proposal $p' \neq p$ in view v . There must exist at least $n - f$ replicas who sent a PREP-VOTE for p' in view v . At least $n - f$ replicas sent a PREP-VOTE for proposal p in view v . These two quorums intersect in at least 1 correct replica, a contradiction since a correct replica only sends one PREP-VOTE message in a view. $\square$

Lemma 2. *If a correct replica commits a (consensus) proposal p in view v , the no correct replica commits a proposal $p' \neq p$ in view v .*

Proof. Suppose for the sake of contradiction a correct replica commits proposal $p' \neq p$ in view v . There must exist a fast COMMITQC or a PREPAREQC for p' in view v . By lemma 1, however, any PREPAREQC or fast COMMITQC in view v must be for proposal p . $\square$

Next, we show that if a proposal committed in some view v , no other proposal can be committed in a future view.

Lemma 3. *If a correct replica commits a consensus proposal p in view v , then any valid PREPARE for view $v' > v$ must contain proposal p .*

Proof. We prove by induction on view v' .

Base case: Let $v' = v + 1$. Suppose for the sake of contradiction there exists a valid PREPARE for a proposal $p' \neq p$ in view v' . There are two cases: either 1) p was committed in view v on the fast path or 2) p was committed in view v on the slow path.

1) Fast path. Since p was committed on the fast path there must exist n PREP-VOTE messages for p in view v , of which at least $n - f$ correct replicas stored a PREPARE for proposal p locally. Any TC in view v contains $n - f$ TIMEOUT messages, and by quorum intersection at least $n - 2f$ of these TIMEOUT messages must contain a PREPARE message for proposal p . Since a TIMEOUT message can contain at most 1 PREPARE message, there cannot exist $n - 2f$ PREPARE messages for p' in view v . By lemma 1, any PREPAREQC or COMMITQC in view v must be for p . Therefore any PREPAREQC, COMMITQC, or set of $n - 2f$ PREPARE messages for p' must be in a view

$< v$. Therefore, the winning proposal for any TC in view v must be p , a contradiction since by the assumption there was a valid PREPARE containing a valid TC for proposal p' .

2) Slow path. Since p was committed on the slow path there must exist $n - f$ CONFIRM-ACK messages (COMMITQC) for p in view v , of which at least $n - 2f$ correct replicas stored a PREPAREQC for proposal p locally. Any TC in view v contains $n - f$ TIMEOUT messages, and by quorum intersection at least 1 of these TIMEOUT messages must contain a PREPAREQC for proposal p . By lemma 1, any prepare or COMMITQC in view v must be for p . Therefore any PREPAREQC or COMMITQC for p' must be in a view $< v$. Therefore, the winning proposal for any TC in view v must be p since even if there does exist $f + 1$ PREPARE for p' in view v , a prepare/COMMITQC takes precedence. This, however, is a contradiction since by the assumption there was a valid PREPARE in view v' containing a valid TC with a winning proposal p' .

Induction Step: We assume the lemma holds for all views $v' - 1 > v$, and now prove it holds for view v' . Suppose for the sake of contradiction there exists a valid PREPARE for a proposal $p' \neq p$ in view v' . Since $v' > v$, this PREPARE message must contain a TC for view $v' - 1$. This TC consists of $n - f$ TIMEOUT messages, for which the winning proposal is p' . This means that the highest (by view) PREPAREQC or set of $n - 2f$ matching PREPARE messages must be for p' . By the base case and induction step any valid PREPARE (and thereby PREPAREQC) in views $> v$ must be for proposal p . There are two cases: either p was committed on the fast path or on the slow path in view v .

Fast Path. Since p was committed on the fast path, the only PREPARE message all correct replicas voted for in view v was for proposal p , so there can exist at most f TIMEOUT messages containing a PREPARE in view v for proposal p' . Thus, any PREPAREQC or set of $n - 2f$ PREPARE for proposal p' must be in a view $< v$. Since p was committed on the fast path, there exists at least $n - f$ replicas who updated their highest PREPARE to be PREPARE for proposal p in view v . By quorum intersection, any TC must contain at least $n - 2f$ of these messages, a contradiction since the winning proposal is for p' .

Slow Path. By lemma 1, any PREPAREQC in view v must be for proposal p . Thus, any PREPAREQC must be in a view $< v$. Since p was committed on the slow path, there must exist at least $n - f$ replicas who voted for a PREPAREQC for proposal p in view v . By quorum intersection, any TC must contain at least 1 TIMEOUT message from this set of replicas, which will contain a PREPAREQC with view at least v . By the induction assumption any PREPARE message with view $> v$ must be for proposal p . Any ties in the view between a PREPAREQC and a set of $f + 1$ PREPARE is given to the PREPAREQC, so the winning proposal must be p for any TC, a contradiction. $\square$

Lemma 4. *If a correct replica commits a consensus proposal in slot s , then no correct replica commits a different consensus proposal in slot s .*

Proof. Let view v be the earliest view in which a correct replica commits a proposal p . Suppose for the sake of contradiction a correct replica commits a proposal $p' \neq p$ in view v' . There are two cases: 1) $v' = v$ and 2) $v' > v$. **Case 1.** By lemma 2 no correct replica will commit a proposal $p' \neq p$ in view v , a contradiction. **Case 2.** Since p' is committed by a correct replica, there must exist a COMMITQC or a fast COMMITQC for p' in view v' , which implies there must exist a valid PREPARE message for p' in view v' . By lemma 3 any valid PREPARE message in view $v' > v$ must be for proposal p , a contradiction. $\square$

Finally, we show that all correct replicas arrive at a consistent log.

Theorem 1. *All correct replicas commit the same requests in the same order.*

Proof. Correct replicas order all consensus proposals by increasing slot numbers. By lemma 4, for any slot s , correct replicas commit the same consensus proposal in slot s . By the collision resistance property of hash functions, all correct replicas will agree on the same history for each tip within a consensus proposal. And since all correct replicas use the same deterministic function that resolves forks in Byzantine lanes, zips the data, and constructs an ordering within a slot, all correct replicas will order the same blocks in the same order. $\square$

A.2 Liveness

Next, we prove that all correct clients' requests will eventually be committed. We assume a continuous input stream of client requests to all correct replicas (and thus that all data lanes progress). We first prove that Autobahn consensus itself is guaranteed to make progress given synchrony, and then show that every correct replica's data proposal will eventually be committed.

To do so, we lay some brief groundwork. We define GST to be the (unknown) time at which the network becomes synchronous, and Δ to be the (known) upper bound on message delivery time after GST. For simplicity we assume synchrony lasts forever after GST⁸. We set a replica's local consensus timeout value for any slot s to 10Δ ; this, we show, suffices to guarantee commitment in good intervals.

We begin by showing that every correct replica will eventually commit a slot s

To do so, we first prove that after GST, a correct leader will take a bounded amount of time to send a PREPARE message. For any view $v > 0$, this is trivial but for $v = 0$ it may take time for a correct leader to satisfy the coverage rule. We show this takes at most 2Δ after GST.

⁸In practice, synchrony is of course finite, and only needs to be "sufficiently" long.

Lemma 5. *If slot s , view v , starts after GST and is led by a correct leader, it will take at most 2Δ time after receiving a ticket to send a PREPARE message in slot s , view v .*

Proof. There are two cases: 1) $v = 0$ or 2) $v > 0$.

Case 1. For view 0, a correct leader must have received a PREPARE message in slot $s-1$ containing a proposal with n tips, of which at least $n-f$ are from correct replicas. A proof of availability for correct tips will take at most Δ time to form after GST, and the next tip at the next greater height will take at most Δ time to reach all replicas. Therefore, by 2Δ , the leader will receive at least $n-f$ new tips, satisfying the coverage rule to propose a new cut.

Case 2. For view $v > 0$, a correct leader must have received a TC for view $v-1$ in slot s . If the winning proposal for a TC is $\neq \perp$, then the leader will immediately send a PREPARE message for slot s , view v , containing the winning proposal (and the TC). Otherwise if the winning proposal is $\perp$, the leader will immediately send a PREPARE message containing its own local set of n tips as the consensus proposal. Therefore, it takes at most 2Δ for a correct leader to send a PREPARE message in slot s , view v . $\square$

Next, we show that all correct replicas will reliably enter a common view, in bounded time. This ensures that all correct replicas will accept messages from the leader, and respond accordingly.

Lemma 6. *For any slot s , let v be a view after GST with a correct leader, and t be the time the first correct replica enters v . All correct replicas will enter v by time $t + 4\Delta$.*

Proof. The earliest correct replica to enter view v in slot s must have received a PREPARE message in slot $s-1$ or a TC in slot s , view $v-1$ (ticket). It will then forward this ticket to the leader of slot s , view v , which will arrive at the leader within Δ time. If $v = 0$, by lemma 5 the leader's proposal will take at most 2Δ time to form. Otherwise if $v > 0$, then the leader will then send a PREPARE message for slot s , which arrives at all correct replicas within Δ time, after which all correct replicas enter view v . $\square$

Recall, that replicas only vote for a consensus proposal (PREPARE) if they can locally assert to the availability of all data. This is trivial when proposing only certified tips (no synchronization is necessary), but when proposing optimistic tips, replicas may need to first fetch missing data. We show that this takes only a bounded amount of time.

Lemma 7. *If a correct replica receives a PREPARE message in slot s , view v , from a correct leader after GST, it will take at most 2Δ before it sends a PREP-VOTE message.*

Proof. a correct replica will only vote for a PREPARE message if it can assert the availability of every tip (and it's history) in the consensus proposal. If the tip is certified (PoA), no further work is necessary, and the replica can vote immediately.

If it instead is not certified (optimistic) a replica must ensure that it has already received the associated data proposal. If it has, it may vote immediately. Otherwise, a replica will send to the leader a SYNC message, requesting all missing optimistic tips, which will arrive at the leader by Δ time. A correct leader, by design, will only propose optimistic tips for which it has all the associated data proposal; thus it will reply to a sync request with the missing data proposal (batch + parent PoA), which takes at most Δ time to arrive. Notably, no sync is necessary for ancestors of the respective tips, as the parent PoA transitively proves it's availability. $\square$

Next, we show that all correct replicas will commit in a view led by a correct leader.

Lemma 8. *If there is some view v in slot s that is led by a correct leader after GST, then all correct replicas commit the correct leader's proposal in view v .*

Proof. By lemma 6, all correct replicas will receive a PREPARE message in slot s , view v' , and enter view v' within 4Δ of the first correct replica entering v' . By lemma 7, all correct replicas will take at most 2Δ time to send a PREP-VOTE to the leader, who will receive these votes by Δ time and form a PREPAREQC from these $n-f$ votes (or a fast COMMITQC if it receives n votes). The leader will then send a CONFIRM message if it formed a PREPAREQC or a COMMIT message if it formed a fast COMMITQC to all replicas, which will arrive within Δ time. If a correct replica receives a valid fast COMMITQC, it will commit. Otherwise, correct replicas will send a CONF-VOTE to the leader, which will arrive at the leader by Δ time. The leader will form a COMMITQC from $n-f$ CONF-VOTE messages, and then send a COMMIT message containing the COMMITQC to all correct replicas, who will receive it by Δ time. Upon receiving a COMMIT message, a correct replica will commit. Since the local timeout is 10Δ , all correct replicas will commit in view v' before timing out. $\square$

Using the above lemma, we show that every correct replica will eventually commit in slot s .

Lemma 9. *Let view v in slot s start after GST. Every correct replica eventually commits a proposal in a view $v' \geq v$ in slot s .*

Proof. Since the number of Byzantine replicas is bounded by f , there exists some view $v' \geq v$ in slot s , such that v' is led by a correct leader. By lemma 8, all correct replicas will commit the correct leader's proposal in view v' . $\square$

To construct the log, a replica must be able to synchronize on all data proposals subsumed by a consensus proposal cut.

Lemma 10. *For any committed proposal p , a correct replica will be able to retrieve all data included (and transitively referenced) by the tips in p .*

Proof. We distinguish two cases: (i) The protocol is instantiated to use only certified tips for proposals, and (ii) the protocol is configured to use the optimistic tip optimization presented in §5.5.2.

(i): Since proposed tips are certified (PoA exists and is provided) at least one correct replica must be in possession of the tip's data proposal *and* all of its history (FIFO voting). A replica can thus directly synchronize on the entire tip history by contacting the quorum of $f+1$ replicas that certified the tip; at least one correct replica will reply.

(ii): A correct replica does not vote for a proposal unless it is in possession of the data for all proposed tips. Since a correct replica only commits upon receiving n PREP-VOTE messages (fast COMMITQC) or $n-f$ CONFIRM-ACK messages (COMMITQC), there must exist at least $f+1$ correct replicas who must have voted. Consequently, for every committed optimistic tip $\geq f+1$ correct replicas have stored the corresponding data. A replica missing a committed optimistic tip thus simply requests to sync from the Quorum of replicas that voted to commit. We remark that optimistic tips are only considered valid by correct replicas if their direct parent is certified. Since the parent is certified, it follows from (i) that the entire tip history can be synchronized. $\square$

Finally, we show that all data proposals from correct data lanes are eventually committed. It follows that all correct client requests are eventually committed and executed:

Lemma 11. *If a correct replica receives a client request it will include it in a car c , and all correct replicas will eventually add a tip subsuming c to its local set of tips.*

Proof. When a correct replica receives a client request it will add it to its next data proposal, which becomes the new tip of its data lane. Before disseminating the data proposal, a correct replica must wait for a proof of availability (PoA) to form for its parent. All messages sent by correct replicas will eventually arrive at all other correct replicas. Since Autobahn's data dissemination layer is free of timeouts, message exchanges simply run at the pace of the network. A correct replica's data proposal will eventually reach all correct replica, and thus $\geq f+1$ correct replicas will vote for it, guaranteeing that a PoA can be formed (and thus allowing the next car to begin). When instantiated with optimistic tips, all correct replicas will adopt the data proposal as this lane's highest optimistic tip. When instantiated without, the proposing replica will form a PoA and disseminate it. All correct replicas will adopt the certified data proposal as the highest tip. Since correct replicas never equivocate their lanes never fork. Thus the tip transitively subsumes all previously disseminated cars. $\square$

Lemma 12. *Given a continuous stream of client requests, a correct replica's lane will grow infinitely.*

Proof. As established in lemma 11, correct replicas will form new cars upon receiving client requests, and succeed in forming PoAs. Since we assume clients to submit a continuous (infinite) stream of input requests, correct replicas' lanes too will grow continuously. $\square$

Lemma 13. *All data proposals of a correct replica's lane will be committed and executed at all correct replicas.*

Proof. There is an infinite number of available consensus slots. Under round robin assignment, there thus must be an infinite amount of correct replicas that are chosen as initial proposer for a slot. It follows from lemma 11 that every correct proposer will eventually see, and thus propose, the highest lane tips from all correct replicas' lanes. By lemma 12 correct lanes grow infinitely, and thus lane coverage will always become satisfied. Finally, since a correct replica never forks, any tip in a correct replica's lane transitively subsumes all cars the replica ever proposed.

By lemma 8 all correct leader proposals after GST will be committed. It follows that all cars in correct lanes will be committed. By lemma 10 all committed data proposals can be retrieved by every correct replica. It follows that all correct replicas will commit and execute all data proposals proposed by correct replicas. $\square$

Theorem 2. *Eventually any client request will be ordered and executed.*

Proof. We assume that each client request will be resent to a different replica if it has not received a response after some timeout. Eventually, some correct replica, h , will receive the client request and include it in a car. By lemma 13, all correct replicas' data proposals will be ordered and executed. $\square$

A.3 Seamlessness discussion

A.3.1 Autobahn is seamless Next, we illustrate that Autobahn is not only live, but also *seamless*. Three design considerations are key to avoid hangovers:

(i) First, Autobahn allows data dissemination to proceed at the pace of the network despite consensus blips. While consensus may fail to make progress due to missed timeouts, data lanes keep growing.

(ii) Second, upon eventual return of a good interval, Autobahn guarantees that all certified disseminated proposals are swiftly committed: consensus proposals reference an arbitrary amount of disseminated proposals with "constant" cost (cost independent of the backlog size)⁹, and missing data can be synchronized with constant protocol latency.

⁹Data proposals in Autobahn are linear in the number of lanes. However, for each lane, only a single tip is needed to reference an arbitrarily long history.

(iii) Finally, asynchronously disseminated data can be fetched off the timeout-critical¹⁰ path of consensus, and thus does not make the protocol more susceptible to violating timeouts.

Our proof of seamlessness does *not* extend to the optimistic tip optimization presented in §5.5.2. We provide some additional discussion at the end of this section.

A note on Lane Coverage: For simplicity, we ignore considerations for lane coverage in the following discussion. We assume that lane coverage is set reasonably (w.r.t to the given dissemination pace), and assume that after the end of a consensus blip lane coverage will always be satisfied for new consensus proposals led by correct replicas. This assumption is justified if data dissemination outpaces consensus, which is "trivially" the case in presence of a consensus blip. If the assumption does not hold, then there cannot exist a noteworthy backlog (w.r.t to lane coverage configuration), and thus we need not be concerned about hangovers.

We first show that data dissemination is responsive, *i.e.*, continues to make progress at the pace of the network. Put differently, Autobahn continues to disseminate client requests even in the presence of consensus blips.

Proof. This follows directly from Lemmas 11 and 12. $\square$

We say that data proposal has been successfully disseminated if it has been certified, and the certificate has reached at least one correct replica.¹¹

We showed in Lemma 11 that all disseminated data proposals will be subsumed by tips once available to a correct replica. Since Autobahn consensus proposals consist of tips, and tips transitively reference the entire lane history it follows:

Corollary 3. *Autobahn can propose arbitrarily long lane histories with constant cost.*

We show next that available tips will be proposed as soon as a correct replica (that has seen the tip) becomes the leader for a new *fresh* slot. Fresh denotes a slot that is in view 0, *i.e.*, for which the leader can propose a new tip cut of choice (we assume lane coverage is given). This is as early as any protocol can hope to propose disseminated data: if it has not arrived at any correct replica, then it is not visible (not successful dissemination), and we cannot expect it to be proposed.

Lemma 14. *All data proposals issued by correct replicas, that have successfully been disseminated to a correct replica R , will be proposed in the first available fresh slot s led by R .*

Proof. A correct replica R that is the leader for a fresh slot s , *i.e.*, $v=0$, will propose the latest available tips it has received

¹⁰Every action, or delay, that may contribute to the violation of a timeout is *timeout-critical*.

¹¹With the optimistic tip optimization it is enough for just the data proposal (uncertified) to arrive at a correct replica.

from all replicas. Consequently, it will implicitly propose to commit *all* data proposals transitively referenced by the lane history. Since correct replica lanes do not fork, this implies that all data proposals issued by correct replicas, that have been successfully disseminated, will be proposed. $\square$

From our liveness proof it follows that the proposal is guaranteed to be committed after GST.

In Lemma 10 we showed that eventually a replica will be able to retrieve all data required to commit. We remark that synchronization for certified tips is asynchronous, *i.e.*, off the timeout-critical path for consensus. Thus Autobahn’s synchronization does not make the consensus protocol more susceptible to blips. Replicas can vote without having synchronized, but must wait to synchronize before locally ordering and executing committed requests.

Next, we show that Autobahn is not only guaranteed to be able to fetch all data, but will do so in constant number of message exchanges (2 total), allowing replicas to complete synchronization by the time they locally commit a slot s (subject to data bandwidth constraints). This property allows Autobahn to be seamless, as the maximum number of message exchanges for synchronization is independent of the size of the backlog. This ensures that synchronization does not introduce protocol-induced delays to commitment. We note that, of course, synchronization speed is bound to the available network bandwidth; we discuss in A.3.2 the implications.

Lemma 15. *For any committed proposal, a correct replica will be able to synchronize on missing cars in constant time, off the timeout-critical path, and by the time it commits the proposal.*

Proof. To synchronize on a certified tip a replica sends a SYNC message to quorum of the $f+1$ replicas who voted to create the proof of availability (PoA), of which at least 1 must be correct. This SYNC message will arrive at this correct replica Δ time afterwards. Since correct replicas respect FIFO voting, this correct replica must have the data for all ancestors for the missing tip. It will then send a SYNC-REPLY message to the requesting replica containing all data for the missing tip and its ancestors. This message will arrive at the correct replica within another Δ time, taking in total 2Δ for a correct replica to synchronize. A correct replica starts synchronizing upon receiving a PREPARE message, and commits at the earliest after 2 more message delays (Fast Path), and latest after 4 more message delays (Slow Path). Since synchronization requires 2 message delays all correct replicas will synchronize on the data *roughly* by the time consensus finishes: $\square$

We conclude that Autobahn is seamless. Consensus blips do not halt data dissemination, and upon return of a good interval, the entire backlog of disseminated data can be committed in one consensus proposal, with protocol complexity independent of the backlog, thus avoiding hangovers.

A note on Optimistic Tips: The optimistic tip optimization may introduce synchronization on the timeout-critical path of consensus, and is thus, according to our definition, not seamless. Nonetheless, we posit that it is, in practice, quite robust to blips as the amount of on-critical-path synchronization is limited to a single car per lane. Replicas need only ever synchronize on the data proposal associated with the optimistic tip before voting. Since the ancestor of an optimistic tip is certified, synchronization of the remaining (possibly long) lane history is asynchronous.

If a correct replica is missing data for an optimistic tip, then by lemma 7 it will take at most 2Δ to retrieve the data. Thus, in practice, one may simply increase the timeout by a constant amount (2Δ) to avoid blips caused by synchronization.

A.3.2 Practical considerations, and non-protocol induced hangovers. Synchronization requires only two message exchanges (a sync request, and a sync reply) to synchronize on arbitrarily long data lanes.

Asymmetric Latencies: Due to assymetric latencies, the 2md required to synchronize may be slightly longer (or shorter!) than the 2md to commit depending on which replicas need to be contacted. Likewise, on the Slow Path, the fastest $n-f$ replicas may create and distribute a *CommitQC* before the slowest f have received PREPARE and started to synchronize. However, at least $f+1$ correct replicas will have synchronized in time successfully, which is enough for output commit. Such practical considerations are orthogonal to seamlessness: any resulting delays are (i) not protocol-induced (not unique to Autobahn and not caused by its protocol mechanisms), and (ii) of little practical concern. Since delays are (i) independent of the size of committed history the effects are exceedingly minor.

Bandwidth Constraints: The size of the sync reply is, of course, proportional to the length data lane and the amount of data that must be fetched. If network bandwidth is a limiting factor, then synchronization of a large history may implicitly cause a brief hangover. We submit that this not in conflict with seamlessness, as it is not consensus protocol-induced. All data must be disseminated at some point, and synchronization does not introduce more data dissemination than was originally intended, and beyond what a subset of correct replicas have already received. Synchronization simply catches up all correct replicas on data dissemination they should have completed but have not, *e.g.* due to volatile network behavior such as partitions; such synchronization is unavoidable. To minimize synchronization in less extreme network cases, one may employ forwarding schemes (*e.g.* gossip of disseminated data) to maximize the amount of data already available. Further, large histories that need to be synced can be staggered, and sent in FIFO order at the bandwidth the network allows. Autobahn may commit proposals as they arrive, and does not need to wait for “all” of a history to arrive in order to start committing. For example, for a lane

l , data proposals at position s can already be ordered and executed before the data proposals for $s+1$ arrive.

Traditional BFT protocols avoid "synchronizing" many messages at once because it does not allow dissemination to advance beyond consensus. They end up disseminating the same amount of data eventually, but will grind to a halt completely in the presence of consensus blips. Notably, replicas not part of a vote quorum may have to synchronize on missing data too upon commit.

A.3.3 Are DAGs seamless? In the following, we briefly discuss the seamlessness of existing DAG protocols. We distinguish two classes of DAG protocols: (i) Certified DAGs, such as Narwhal [26] or Bullshark [54] employ Reliable Broadcast (RB) in each round to disseminate data. (ii) Uncertified DAGs, such as Cordial Miners [44] or BBKA-Chain [47] employ only Best-Effort-Broadcast (BEB) for data dissemination. We note that these two designs are currently theoretical-only, and have not been empirically evaluated.¹²

Synchronization on timeout-critical path. All four of the above DAG protocols incur blocking synchronization on the timeout-critical path of consensus. For uncertified DAGs this is fundamental. Since data dissemination is best effort, edges in DAG nodes simply consist of reference (hash digest) to a preceding (typically of the previous round¹³) data proposal. Analogous to optimistic tips in Autobahn (§5.5.2), such uncertified edge references do not guarantee availability. Since a replica can only ascertain the availability of an edge by locally being in possession of the data, it must synchronize on missing edges before adopting (or voting for) a new DAG node to assure liveness. Such synchronization is thus on the timeout-critical path of consensus, and makes the protocol susceptible to violating timeouts. Surprisingly, we observed existing certified DAGs (specifically Narwhal and Bullshark) also perform synchronization on the timeout-critical path. Existing protocol descriptions (Bullshark [54], Alg.2, L.52) and open-source implementations [33] require causal histories to be available locally *before* voting. Consequently, synchronization may contribute to producing consensus blips. However, synchronizing on the timeout-critical path is not fundamental to certified DAGs. Round certificates ($2f+1$ votes) implicitly act as availability proofs (akin to PoA in Autobahn). Thus replicas can be allowed to vote before synchronizing, and data fetching can be moved off the timeout-critical path (in the background).

Recursive synchronization. In DAGs, each node contains edges to *some* subset of preceeding nodes. Typically, a node in round r requires $2f+1$ references to nodes in round $r-1$. Because of this design, it is not possible to infer the full

causal history from a single DAG node in constant time. For instance, given only a node in round r , it is impossible to infer which other replicas' nodes it referenced in some round $r-k$ ($k > 1$). The causal history may be "arbitrarily" intertwined, and can only be inferred by recursively tracing back the path of the edges. As a result, in order to synchronize on a node's history, a replica may need to issue several rounds of sequential data fetching, in each round learning more of the history and thus (i) what needs to be fetched next, and (ii) from which replica to fetch it (there is no guarantee that one correct replica is in possession of the entire required history). Unfortunately, such unpredictable synchronization time may result in hangovers.

We speculate that Narwhal and Bullsharks choice to synchronize on the timeout-critical path is largely to avoid such behavior. In practice, synchronizing on the timeout-critical path may help Narwhal and Bullshark to streamline synchronization and make commit time more predictable.¹⁴ This makes the DAG operate somewhat in lock-step (and grow slower), but still cannot fully sidestep the problem. Furthermore, doing so also results in a conundrum between synchronization on the timeout-critical path, and poor robustness/predictability of synchronization time.

Our communications with companies deploying DAGs [7] support our claims. They report that in heterogeneous deployments (which are the norm in decentralized systems), replicas frequently incur large amount of synchronization (on the timeout-critical path), resulting in timeouts and loss of consensus progress. To avoid such scenarios, recent work has focused on leveraging reputation schemes [53] to try to elect as leaders only the fastest, and best connected replicas. While this can help to some extent in practice, it does not eliminate the fundamental shortcoming; truly seamless protocols, like Autobahn, provide a more principled solution. Autobahn's lane based design, for instance, drastically simplifies reasoning about synchronization: it easy to infer what needs to be synced on (a lane prefix), and where to request it from (a single correct replica must have it all).

A.4 Fork Garbage Collection and Bounded Wastage

Under round-robin leader assignment every correct leader eventually issues a proposal. Consequently, every certified data proposal (*i.e.*, PoA) that has been received by any correct replica will eventually be proposed, and thus committed (reliable inclusion). This guarantees liveness for correct replica proposals, but it also allows us to (i) garbage collect forks, and (ii) bound the amount of data proposals a Byzantine replica can disseminate without intending to commit.

In order to issue a new data proposal, a Byzantine replica must certify it's previous car and distribute the PoA. By the reliable inclusion property, a Byzantine replica thus cannot

¹²BBKA-Chain uses a PBFT-like primitive for leader DAG nodes. All other DAG nodes use BEB.

¹³Most DAGs progress in a rigid round by round pattern, but some exceptions exist; BBKA-Chain, for instance, appears to not prescribe any particular structure to its non-leader nodes.

¹⁴In Cordial Miners correct replicas pessimistically forward locally received data to other replicas. This helps improve state homogeneity, and thus reduces need for sync, but introduces potentially redundant overheads.

issue a new data proposal, without implicitly confirming to commit a data proposal at all prior positions. In short, it is guaranteed that a data proposal will be committed for all but the latest lane position pos_{latest} .

We note, that this does not guarantee that all certified data proposals with $pos < pos_{latest}$ will commit. Since Byzantine replicas may equivocate, it is possible to produce PoA's for several different proposals per position. Each correct replicas votes for (and stores) at most one data proposal per lane position, but different replicas may store different certified data proposals for the same position.

By the reliable inclusion argument, all positions $< pos_{latest}$ will eventually be committed. Commit will order at most one proposal per position, and replicas can discard all obsolete data proposals locally stored. Thus it is guaranteed that eventually all forks will be resolved and garbage collected.

Furthermore, correct replicas store at most one data proposal issued by a Byzantine replica that is not guaranteed to commit or garbage collected, *i.e.*, the data proposal at pos_{latest} . Since each data proposal enforces a bounded batch size b , a correct replica may at most have to store $f*b$ transactions (one proposal from f Byz Replicas) that will never commit or be garbage collected, thus bounding the amount of *wastage* in the local mempool at any time.

Pipelined cars. Autobahn can, if desired, be augmented to pipeline the certification process of consecutive proposals to minimize sequential wait times in the case that batches are filled exceedingly quickly. Like parallel consensus proposals, the amount of uncertified cars at a given time may be bounded by enforcing that a position p cannot begin before $p-k$ has been certified. Pipelining up to k cars, in turn, allows a Byzantine replica to distribute up to k uncertified proposals that are not guaranteed to be reliably included. The wastage bound follows accordingly: $k*f*b$ transactions might never be committed or garbage collected.

B Additional Technical Discussion

In the following we discuss some optional modification that are not integral to the core Autobahn protocol. We do not implement or evaluate these, but find there is value in nonetheless presenting them.

B.1 Avoiding Synchronization with Optimistic Tips

The optimistic tip optimization presented in §5.5.2 can, in the presence of Byzantine actors, cause (a constant amount of) synchronization on the timeout-critical path. For instance, a Byzantine proposer may forward its batch only to a correct leader, and no other replicas; if the leader proposes the batch, it must aid other correct replicas with synchronization before they may vote, which may potentially cause its view to time out.

To reduce unwanted synchronization in practice, Autobahn can be augmented with a simple (replica-local) reputation mechanism. A leader forced to synchronize on some lane l 's

tip downgrades its local perception of l 's reputability. Below a threshold, the leader stops proposing optimistic tips for l , and simply falls back to proposing a certified tip. Each replica is thus responsible for warranting optimism for its own lane and consequently its own data proposals' latency. Reputation can be regained over time, *e.g.* for each committed car.

We remark that such a reputation scheme serves exclusively to minimize *redundant* dissemination, and is not necessary for safety or liveness. Without, the worst case synchronization effort on the critical path remains constant; only the optimistic tips must be synchronized, *i.e.*, at most one data proposal per lane.

B.2 Ride Sharing

Consensus phases and cars share the same basic message pattern [13] and can, in theory, be sent and signed together (*ride-sharing*) to minimize the total number of messages (and associated authentication costs).

However, it may not always be convenient to hitch a ride: Cars must slow down to accommodate the larger Quorum sizes of consensus, and consensus messages may need to wait for available cars (up to 2md if one narrowly misses a PROPOSAL. Ride sharing thus poses a trade-off between optimal resource utilization and latency.

We find that in practice that, even for small n , consensus coordination is not a throughput bottleneck (data processing is, §6), and thus we opt to evaluate Autobahn without ride sharing. We note that the relative consensus overheads decrease linearly as n scales, and are thus already exceedingly small for $n \gg k$ (max #instances).

Nonetheless, for completeness, we discuss how to theoretically implement ride-sharing in Autobahn.

Shared Cars. Consensus messages can be embedded on the lane of the respective instance leader (Fig 9); other lanes remain unaffected. Prepare and Confirm phases are piggy-backed on a car each while TIMEOUTS may ride any message. Multiple consensus instances (*i.e.*, messages for different slots and views) may hitch a ride on the same car and share one "batched" signature. In exchange, a replica may need to forward all messages in a batch to prove the validity of a single signed message; one can reduce costs by signing a merkle root of the batch, attaching a small merkle path to authenticate individual messages, and caching repeat signatures [58].

Cars, by default, only wait for $f + 1$ votes – too few to successfully assemble PrepareQCs and CommitQCs. To account for this we upgrade any car carrying one or more PREPARE/CONFIRM consensus messages into a special *ambassador* car that waits for $n - f$ votes; ambassadors representing PREPARES may attempt to briefly wait for the full n PREPARE-VOTES required to construct a Fast-CommitQC. Fig. 4 illustrates the embedding, flags denote ambassadors. Correct replicas will always vote to confirm PROPOSAL availability, but may omit a consensus-vote if they have changed views

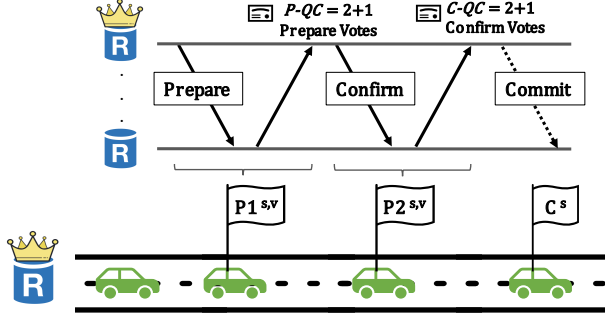


Figure 9. Core Consensus coordination pattern. Messages may be piggybacked on existing cars: P1 = PREPARE, P2 = CONFIRM, C = COMMIT.

for the respective slot. To distinguish faulty voters from correct replicas Autobahn delays cars following an ambassador: ambassador cars wait for the earlier of QC formation, and expiration of a short timer (beyond the first $f + 1$ replies (PoA)). Upon timing out, a lane proceeds with the next car, and continues consensus processing for late arrivals individually or on a later car. We remark that although the data proposal pace may decelerate due to increased wait-time for ambassador cars, lane growth fundamentally remains asynchronous.

Ridin' Solo. It may not always be convenient for consensus messages to hitch a ride. In the worst-case, a consensus message that is available just after starting the current car

may need to wait up to $2md$ until the next PROPOSAL ($1md$ in expectation). Ride sharing thus poses a trade-off between optimal resource utilization and latency. To strike a middle-ground, consensus message may wait for a small timeout to see whether they can hitch a ride, and otherwise drive themselves. However, rather than sending consensus messages fully independently, Autobahn may once again piggyback them alongside data proposals in a special *off-lane car* to maximize message utility.

This off-lane car follows an ambassador car's usual structure, and may carry both multiple consensus instance messages. Uniquely, however, it does not have a lane position, and does not obey lane rules (certified chained FIFO).

In order to minimize end-to-end latency, the off-lane car includes as parent reference the latest proposal (yet to be certified) proposal sent by the leader. This is safe and live as it is equivalent to an optimistic leader tip; the consensus leader is responsible for the delivery of its tip history.

Since the off-lane car lacks a position, normal cars in the leaders lane do not extend the off-lane car. To avoid orphaning an off-lane car whose payload was certified but did not commit, a subsequent car may be augmented to include a special (optional) *tow* reference to a certificate of an off-lane car. This allows the off-lane car to eventually be committed as part of the towing car's history.